

Material de Apoio: Curso Presencial Internet das Coisas com Arduino + ESP01



1ª
EDIÇÃO
revisão 1.0

Internet das Coisas Arduino + ESP8266

Prof. Me. Breno M. de Oliveira
prof.brenooliveira@gmail.com
Julho/2018

AVISOS

Este material foi desenvolvido para auxiliar os alunos do Curso Presencial IoT - Arduino + ESP8266, ministrado pelo Prof. Breno M. de Oliveira no desenvolvimento das atividades práticas. No entanto, essa apostila é gratuita e destinada a qualquer pessoa que deseja aprender mais sobre Internet das Coisas, Arduino e ESP8266. Esperamos que esse material ajude na propagação do conhecimento.

A publicação total e/ou parcial do conteúdo desta apostila para fins lucrativos é proibida. O uso do conteúdo em outros materiais é permitido desde que haja as devidas referências.

Todos as imagens, marcas e modelos de software e hardware utilizados neste curso, possuem seus respectivos direitos autorais.

Arduino: <http://arduino.cc>

Visual Studio Code: <https://code.visualstudio.com/>

Node-RED: <https://nodered.org/>

Ai-thinker : <https://www.ai-thinker.com/>

Espressif: <https://www.espressif.com/>

Esta apostilas poderá sofrer atualizações e correções. Para obter a última versão gratuitamente e ficar por dentro dos cursos e treinamentos oferecidos pelo Prof. Breno M. de Oliveira.

ACESSE:
profbrenooliveira.ddns.net



INDAGAÇÕES INICIAIS

- Quem é o ministrante deste curso?
- Por que aprender sobre internet das coisas?
- Qual a importância dos microcontroladores no cenário atual?
- Vou sair daqui um expert em internet das coisas?
- Vai ter cooffee break?

O QUE SERÁ ABORDADO NESTE CURSO?

- Conceitos essenciais a respeito da Internet das Coisas
- Infraestrutura de Redes / Internet
- Conceitos básicos de eletrônica
- Conceitos básicos e avançados de programação
- Práticas de projetos de hardware embarcado

VAMOS AO TRABALHO: CONCEITO DE INTERNET DAS COISAS

Se vamos apreender técnicas para construção de dispositivos e sistemas para Internet das Coisas, devemos ter em mente exatamente o que esse tipo de projeto deve envolver.

Muitas vezes podemos desenvolver alguma coisa e associar a ele o conceito de Internet das Coisas, mas no fundo estamos apenas usando princípios de comunicação eletrônica.

O CONCEITO É SIMPLES,
PORÉM COMPLICADO!





A ORIGEM DO TERMO IOT...

Kevin Ashton, pesquisador britânico do Massachusetts Institute of Technology (MIT), é considerado o primeiro especialista a usar o termo “Internet das Coisas (IoT, na sigla em inglês), em 1999. Em uma entrevista para o Finep em 2015, questionado sobre a origem do termo, ele respondeu:

Kevin Ashton: Numa apresentação para executivos da Procter & Gamble em 1999, quando eu falava da ideia de se etiquetar eletronicamente os produtos da empresa, para facilitar a logística da cadeia de produção, através de identificadores de radio frequência (RFID, em inglês), na época um assunto novíssimo e “quente”. A expressão “Internet das Coisas” pode nem ser tão brilhante, mas deu um bom título à apresentação, e logo se popularizou. Na verdade, a combinação de palavras foi como o resultado de um insight importante, de algo que ainda é mal compreendido.

MUITO ALÉM DA NOSSA IMAGINAÇÃO...

Internet das Coisas é um termo extenso que cobre uma gama enorme de tecnologias e serviços, ao qual dependendo do caso de uso, pode integrar um ecossistema de tecnologias como:

- ➡ inteligência artificial;
- ➡ computação na nuvem;
- ➡ ciber segurança;
- ➡ análise de dados avançada;
- ➡ Big Data;
- ➡ tecnologias de comunicação digital;
- ➡ realidade virtual;
- ➡ robótica;
- etc.



Arduino! Revendo nosso velho conhecido...

Mas afinal, do que realmente estamos falando?

De um microcontrolador?

De um hardware específico?

De uma linguagem de programação?

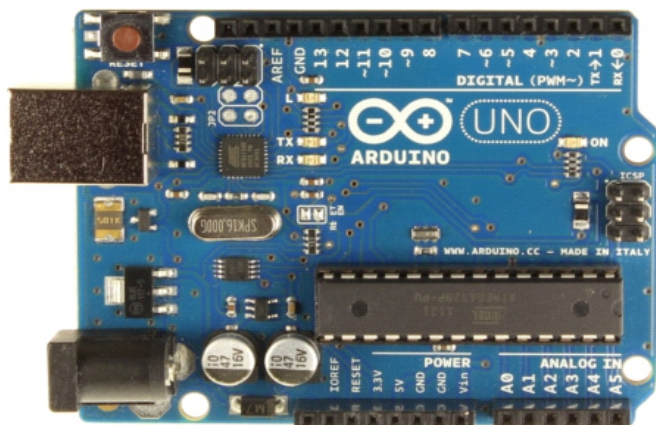
De um ambiente de desenvolvimento?



ESTAMOS FALANDO DE TUDO ISSO AO MESMO TEMPO!



A FAMÍLIA ARDUINO...

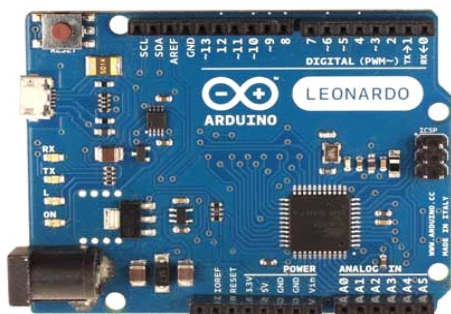


Arduino Uno
www.arduino.cc

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)
Clock Speed	16 MHz

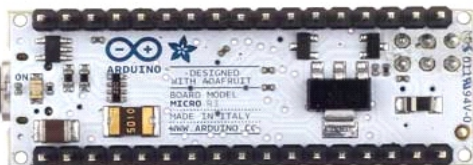


Arduino! Revendo nosso velho conhecido...



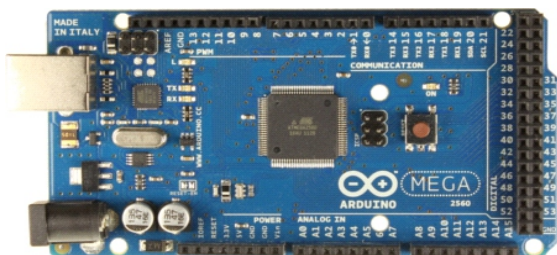
Arduino Leonardo
www.arduino.cc

Microcontroller	ATmega32u4
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	20
PWM Channels	7
Analog Input Channels	12
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega32u4) of which 4 KB used by bootloader
SRAM	2.5 KB (ATmega32u4)
EEPROM	1 KB (ATmega32u4)
Clock Speed	16 MHz



Arduino Micro
www.arduino.cc

Microcontroller	ATmega32u4
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	20
PWM Channels	7
Analog Input Channels	12
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega32u4) of which 4 KB used by bootloader
SRAM	2.5 KB (ATmega32u4)
EEPROM	1 KB (ATmega32u4)
Clock Speed	16 MHz

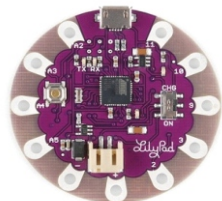


Arduino Mega

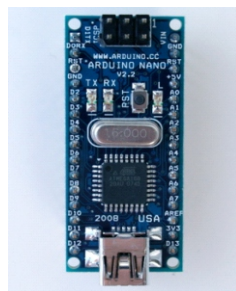
Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	54 (of which 15 provide PWM output)
Analog Input Pins	16
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz



Arduino! Revendo nosso velho conhecido...



LilyPad Arduino USB



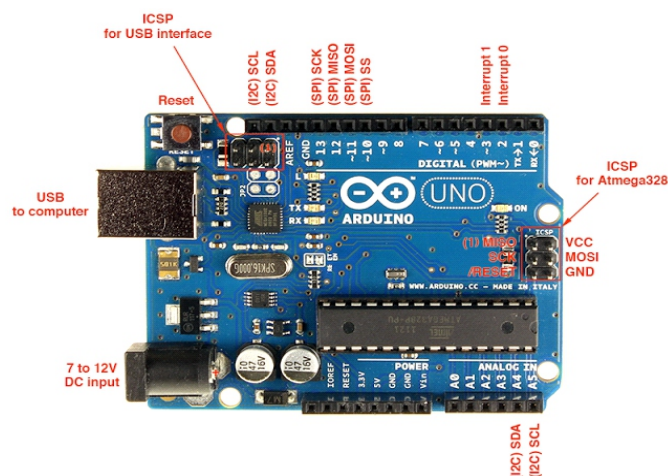
Arduino Nano



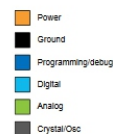
Arduino Pro Mini

Dentre outros...
www.arduino.cc

CARACTERÍSTICA DO ARDUINO UNO R3

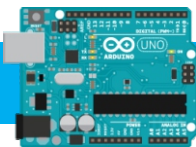


(PCINT14/RESET) PC6	1	28	PC5 (ADC5/SCL/PCINT13)
(PCINT16/RXD) PD0	2	27	PC4 (ADC4/SDA/PCINT12)
(PCINT17/TXD) PD1	3	26	PC3 (ADC3/PCINT11)
(PCINT18/INT0) PD2	4	25	PC2 (ADC2/PCINT10)
(PCINT19/OC2B/INT1) PD3	5	24	PC1 (ADC1/PCINT9)
(PCINT20/XCK/T0) PD4	6	23	PC0 (ADC0/PCINT8)
VCC	7	22	GND
GND	8	21	AREF
(PCINT6/XTAL1/TOSC1) PB6	9	20	AVCC
(PCINT7/XTAL2/TOSC2) PB7	10	19	PB5 (SCK/PCINT5)
(PCINT21/OC0B/T1) PD5	11	18	PB4 (MISO/PCINT4)
(PCINT22/OC0A/AIN0) PD6	12	17	PB3 (MOSI/OC2A/PCINT3)
(PCINT23/AIN1) PD7	13	16	PB2 ($\overline{SS}$ /OC1B/PCINT2)
(PCINT0/CLKO/ICP1) PB0	14	15	PB1 (OC1A/PCINT1)



FONTE: Datasheet ATMEGA328P - Microchip

http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-42735-8-bit-AVR-Microcontroller-ATmega328-328P_Datasheet.pdf



Arduino! Revendo nosso velho conhecido...

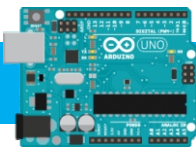
INSTALANDO A IDE DE DESENVOLVIMENTO

1º - Acessar: www.arduino.cc

2º - Clicar em: Software/Downloads

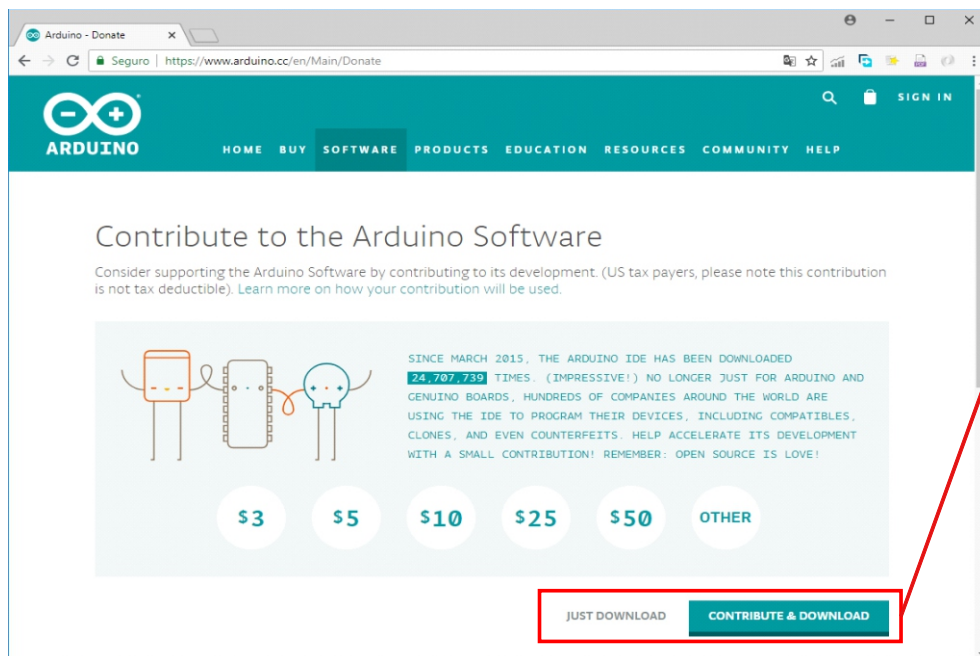
Download the Arduino IDE

3º - Clicar em: Windows: Installer,...

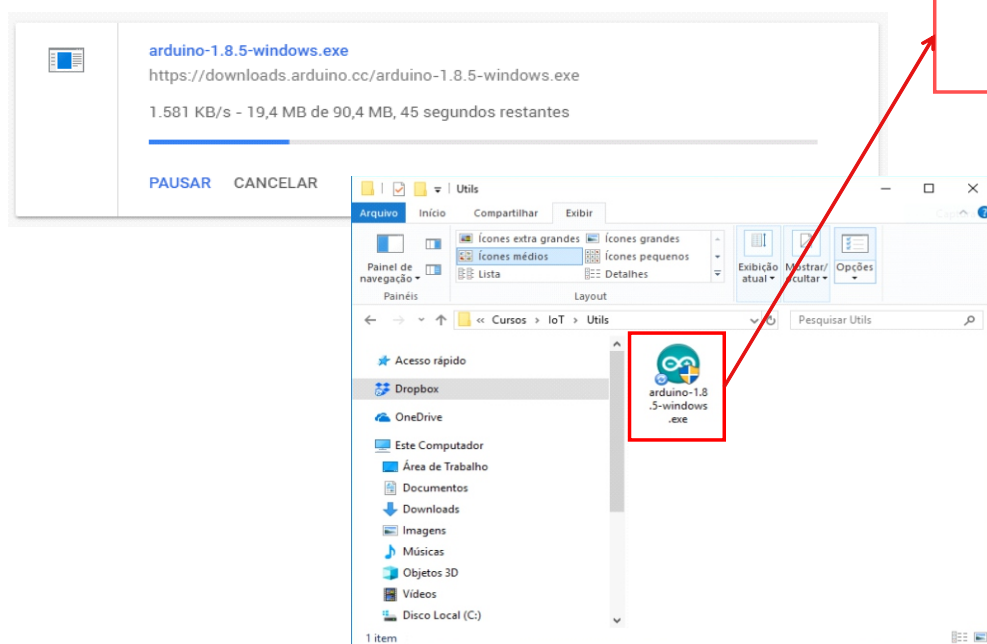


Arduino! Revendo nosso velho conhecido...

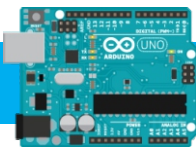
INSTALANDO A IDE DE DESENVOLVIMENTO



4º - Clicar em:
Just Download
Ou
Contribute & Download

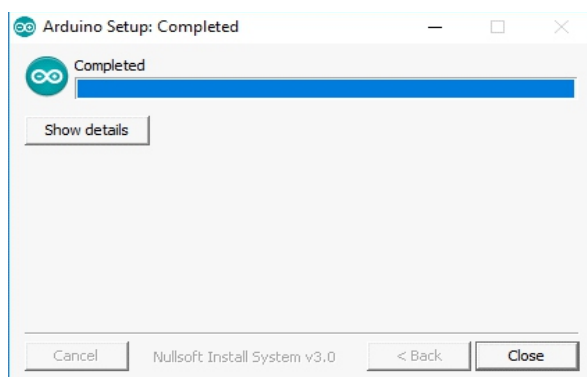
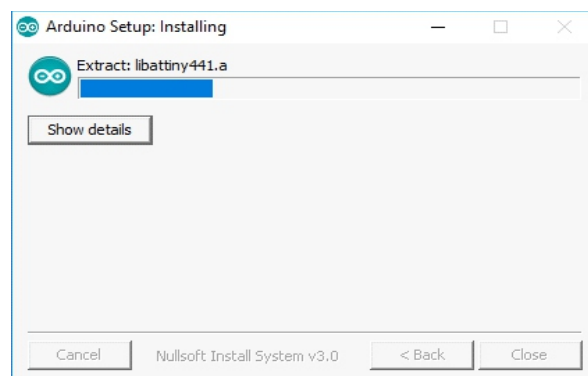
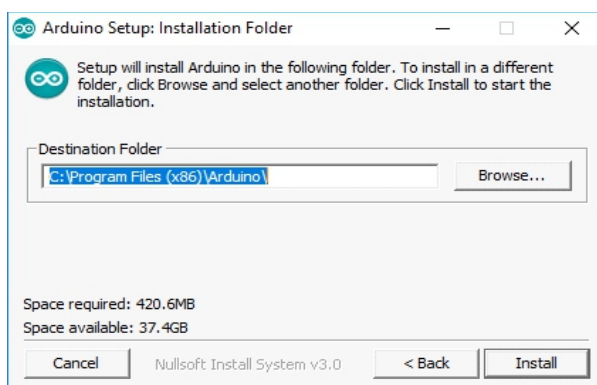
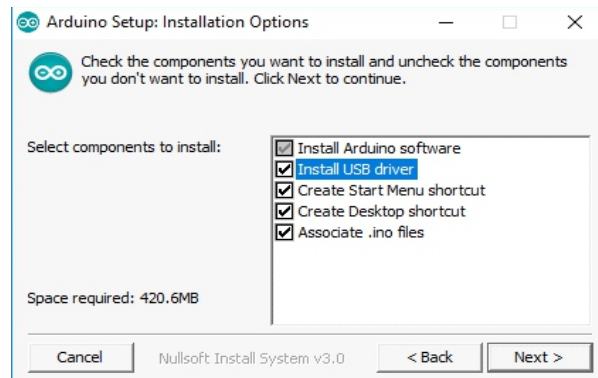
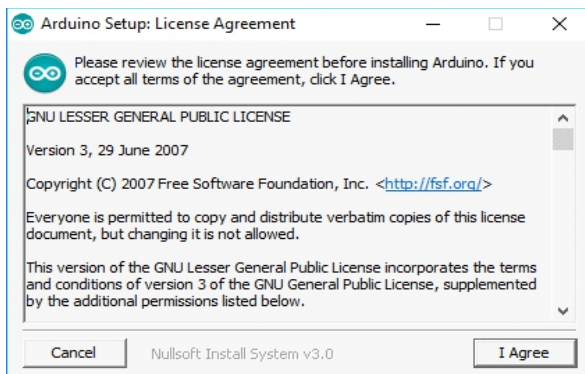


5º - Executar o
Instalador



Arduino! Revendo nosso velho conhecido...

INSTALANDO A IDE DE DESENVOLVIMENTO

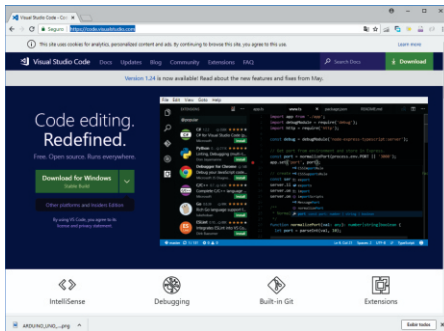


Visual Studio Code, nosso novo companheiro

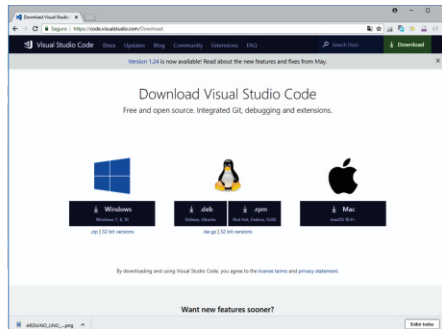
INSTALANDO O VISUAL STUDIO CODE

1º Acesse: <https://code.visualstudio.com/>

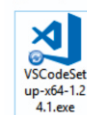
2º Clique em Download



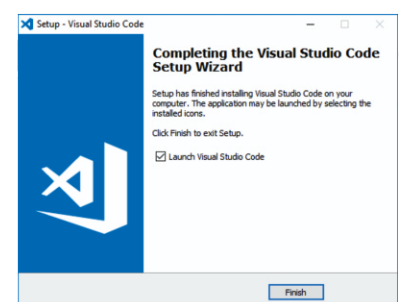
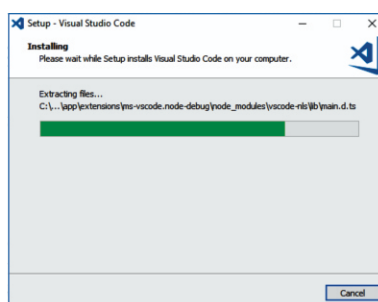
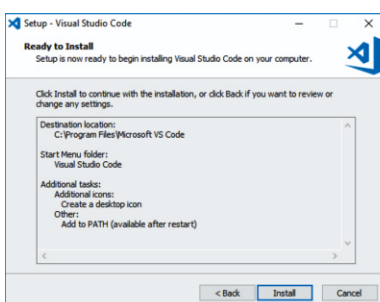
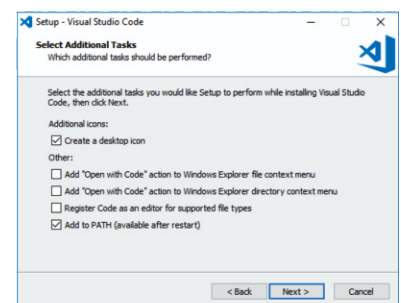
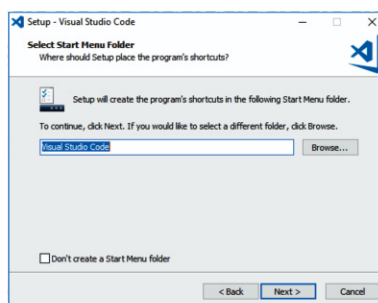
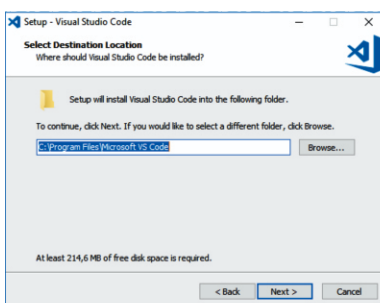
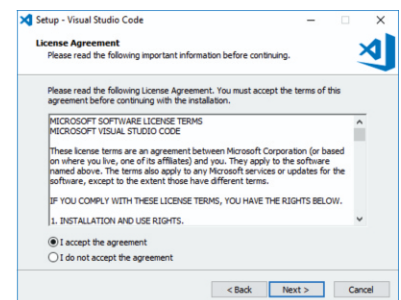
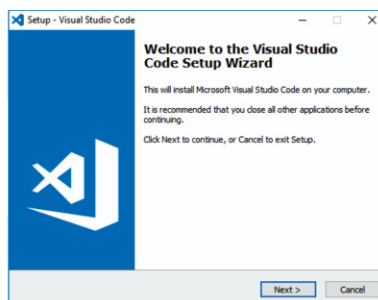
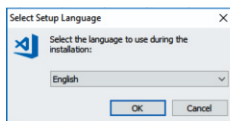
3º Baixe a versão para Windows

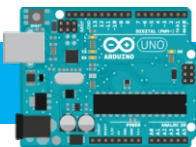


4º Execute o instalador



5º Selecione a linguagem e clique em OK. Continue o procedimento da instalação clicando em Next, Next, Next...



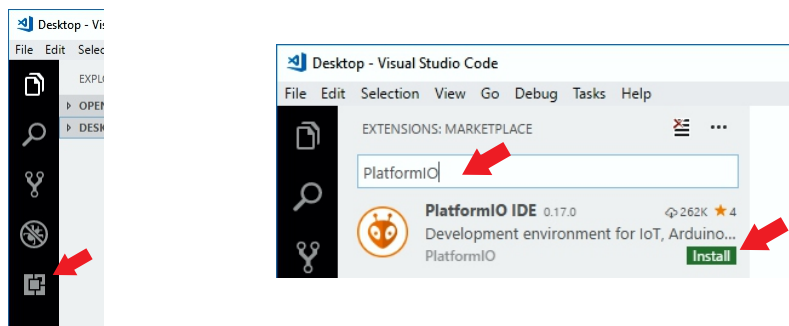


Arduino! Revendo nosso velho conhecido...

INSTALANDO O PLATFORM.IO DENTRO DO VISUAL STUDIO CODE

1º Abra o Visual Studio Code e clique na barra lateral esquerda, no botão "Extensions"

2º Digite no campo de busca: "PlatformIO". Clique em Install.

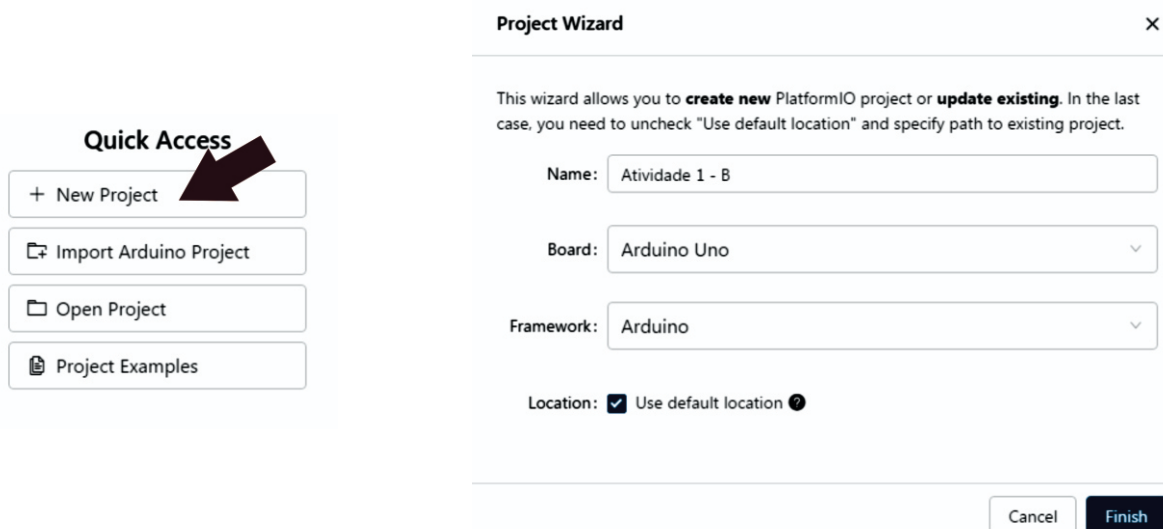


3º Aguarde até a instalação da extensão finalizar e clique em Reload para instalar os complementos.

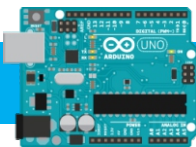
INICIANDO UM NOVO PROJETO PARA ARDUINO NO VS CODE

1º Abra o Visual Studio Code. Uma página inicial da PlatformIO deve ser carregada, conforme a imagem abaixo. Clique em New Project.

2º Preencha o nome do projeto e a placa de desenvolvimento. No nosso caso, Arduino Uno.



3º Clique em Finish. Uma nova estrutura de projeto é criada. A partir de agora é possível criar um projeto para Arduino compilando fora da IDE oficial e com todos os recursos do Visual Studio Code.



A LINGUAGEM DO ARDUINO

Os Símbolos usados na construção de funções em C são os seguintes:

{ } - Dentro das chaves vão os procedimentos (statements) que a função deve executar;

;- O ponto-e-vírgula é usado para marcar o final de um procedimento;

// - comentário de uma linha: qualquer caractere depois das duas barras é ignorado pelo programa;

/*...*/ - comentário em várias linhas: qualquer texto colocado entre esses símbolos também é ignorado pelo programa.

CONSTANTES

CONSTANTES são valores predefinidos que nunca podem ser alterados. Na linguagem C do Arduino são 3 os grupos de constantes; os dois componentes de cada grupo sempre podem ser representados pelos números binários 1 e 0.

TRUE/FALSE: são constantes booleanas que definem estados lógicos. Verdadeiro é qualquer valor que não seja zero. Falso é sempre o valor zero.

HIGH/LOW: essas constantes definem as tensões nos pinos digitais do Arduino. Alto é uma tensão de 5 volts; baixo o terra (ou 0 volt).

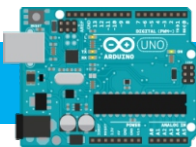
INPUT/OUTPUT: são constantes programadas pela função `pinMode()` para os pinos do Arduino; eles podem ser entradas (de sensores) ou podem ser saídas (de controle).

VARIÁVEIS

São posições na memória de programa do Arduino marcadas com um nome e o tipo de informação que irão guardar. Essas posições podem estar vazias ou podem receber um valor inicial. Os valores das variáveis podem ser alterados pelo programa.

Escopo da Variável é o limite ou abrangência da variável. Uma variável pode ser declarada em qualquer parte do programa. Se for declarada logo no início, antes da função `setup()`, ela tem o escopo de Variável Global, e por isso ela pode ser vista e usada por qualquer função no programa. Se declarada dentro de uma função ela tem o escopo de Variável Local, e só pode ser usada por essa função.

Declaração da Variável como as funções, deve ser declarada antes de ser chamada. Essa declaração consiste em atribuir previamente um tipo e um nome à variável.



TIPOS DE VARIÁVEIS

byte - esse tipo armazena 8 bits (0-255);
int – armazena números inteiros de até 16 bits;
unsigned int - Inteiros positivos somente;
long - armazena números inteiros de até 32 bits;
unsigned long - Inteiros positivos somente;
float - variáveis deste tipo podem armazenar números fracionários de até 32 bits.
double - igual a float no Arduino
Boolean - Verdadeiro ou Falso, 1 ou 0, True ou False
char - caractere (1 byte)
byte - 8 bits

Data Types
String
String()
array
bool
boolean
byte
char
double
float
int
long
short
unsigned char
unsigned int
unsigned long
void
word

SÍMBOLOS COMPOSTOS

Símbolos compostos são aqueles que combinam os símbolos aritméticos entre si e com o sinal de atribuição:

x++ // x=x+1
x-- // x=x-1
x += y // x=x+y
x -= y // x=x-y
x *= y // x=x*y
x /= y // x=x/y

OPERADORES DE COMPARAÇÃO

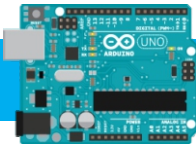
Operadores de comparação comparam uma variável com uma constante, ou variáveis entre si. São usados para testar se uma condição é verdadeira.

x == y // x é igual a y
x != y // x não é igual a y
x < y // x é menor que y
x > y // x é maior que y
x <= y // x é menor ou igual a y
x >= y // x é maior ou igual a y

OPERADORES LÓGICOS

Operadores lógicos são usados para comparar duas expressões, retornam 1 ou 0 (verdadeiro/falso).

&& AND porta lógica 'E'
|| OR porta lógica 'OU'
! NOT porta lógica NÃO



CICLO DE VIDA DE UM SOFTWARE NO ARDUINO

```
void setup() {  
  // Comandos executados somente uma vez.  
  
}  
  
void loop() {  
  // Comandos executados repetidamente.  
  
}
```

CRIANDO SUAS PRÓPRIAS FUNÇÕES

Muitas vezes precisamos desenvolver programas com quantidade significativa de código. Isso acaba se tornando um problema na organização do nosso programa. Para resolver esse problema, o ideal é o programador ir criando seus próprios métodos e funções a fim de tornar seu código mais organizado e flexível. Para criar uma função no Arduino, devemos seguir a seguinte estrutura:

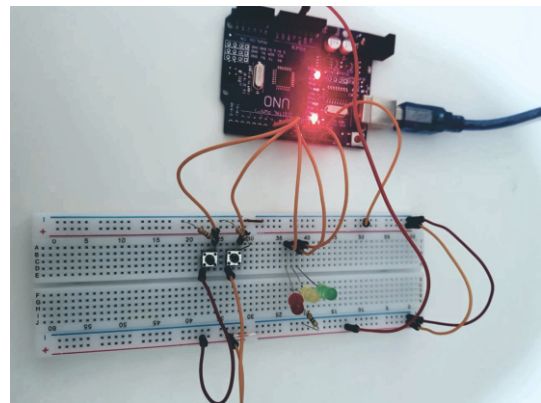
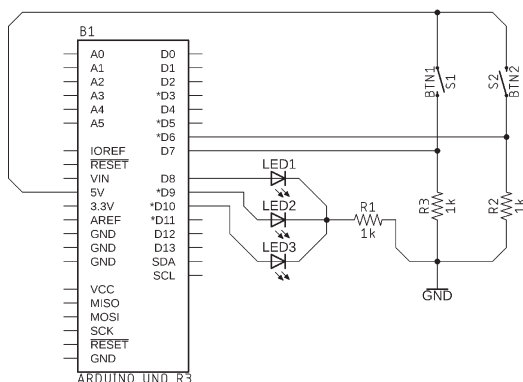
```
//Corpo de um procedimento (não há retorno de valores);  
void NomedaFuncao(parametro1, parametro1) {  
  //Comandos da Função  
}  
  
//Corpo de uma função (há retorno de valores);  
tipo_retorno NomedaFuncao(parametro1, parametro1) {  
  //Comandos da Função  
  return  
}
```



ATIVIDADE 1A - TRABALHANDO COM ENTRADAS E SAÍDAS DIGITAIS

Para iniciarmos as atividades práticas deste curso, vamos abordar uma aplicação bem simples, porém utilizando conceitos novos da programação. Iniciaremos mostrando como acender leds usando a maneira mais clássica e direta na própria IDE do Arduino. Seguem os passos:

1º Monte o esquemático da figura abaixo, utilizando 2 botões (BTN1 e BTN2) e 3 Leds (LED1, LED2 e LED3).



2º Abra a IDE do Arduino, configure sua placa de desenvolvimento e digite o código abaixo:

```
/*
  ATIVIDADE 1 - Trabalhando com entradas e saídas digitais
  CURSO IoT - Julho 2018
  Internet das Coisas com Arduino e ESP-01
  Autor: Prof. Breno M. de Oliveira
  */
//Declarando os pinos dos LEDs
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10
//Declarando os pinos dos Botoes
#define pin_btn1 6
#define pin_btn2 7

void setup() {
  // O código dentro do SETUP só executa um vez.
  pinMode(pin_led1, OUTPUT);
  pinMode(pin_led2, OUTPUT);
  pinMode(pin_led3, OUTPUT);
  pinMode(pin_btn1, INPUT);
  pinMode(pin_btn2, INPUT);
  Serial.begin(9600);
}

void loop() {
  // O código dentro do LOPP executa infinitamente.
  if (digitalRead(pin_btn1) == 1) {
    Serial.println("OK");
    digitalWrite(pin_led1, HIGH);
  }
  else {digitalWrite(pin_led1, LOW);}
  delay(10);
}
```



3º Agora, vamos executar o mesmo programa utilizando o VS Code:

- Copie o código digitado anteriormente para área de transferência (CTRL+C)
- Feche a IDE do Arduino.
- Abra o VS Code, inicia-se um novo projeto na plataforma PlatformIO, salve com o nome de Atividade1A.
- No arquivo main.cpp da pasta source, cole o código anterior e insira a seguinte linha programação, abaixo do cabeçalho do programa.

```
#include <Arduino.h>
```

- O código fonte deverá ficar semelhante ao código abaixo:

```
/*  
ATIVIDADE 1A - Trabalhando com entradas e saídas digitais  
CURSO IoT - Julho 2018  
Internet das Coisas com Arduino e ESP-01  
Autor: Prof. Breno M. de Oliveira  
*/
```

```
#include <Arduino.h>
```

```
//Declarando os pinos dos LEDs  
#define pin_led1 8  
#define pin_led2 9  
#define pin_led3 10
```

```
//Declarando os pinos dos Botoes  
#define pin_btn1 6  
#define pin_btn2 7
```

```
void setup() {  
    // O código dentro do SETUP só executa um vez.
```

```
    pinMode(pin_led1, OUTPUT);  
    pinMode(pin_led2, OUTPUT);  
    pinMode(pin_led3, OUTPUT);
```

```
    pinMode(pin_btn1, INPUT);  
    pinMode(pin_btn2, INPUT);
```

```
    Serial.begin(9600);
```

```
}
```

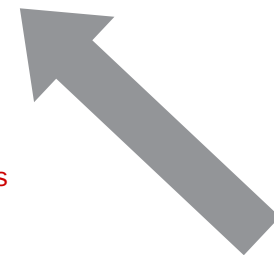
```
void loop() {  
    // O código dentro do LOPP executa infinitamente.
```

```
    if (digitalRead(pin_btn1) == 1) {  
        Serial.println("OK");  
        digitalWrite(pin_led1, HIGH);  
    }
```

```
    else {digitalWrite(pin_led1, LOW);}
```

```
    delay(10);
```

```
}
```



Arduino.h é a biblioteca responsável pelo funcionamento dos comandos da linguagem Arduino dentro na Plataforma/Vs Code.



ATIVIDADE 1B - TRABALHANDO COM ENTRADAS E SAÍDAS DIGITAIS DE UMA OUTRA MANEIRA

Vamos implementar as mesmas funcionalidades do programa anterior, porém utilizando uma abordagem mais elegante. O aluno deve utilizar essas técnicas a fim de otimizar sua programação, deixando-a mais limpa e fácil compreensão. Escreva os códigos abaixo e envie novamente para o Arduino.

```
/*
ATIVIDADE 1B - Trabalhando com entradas e saídas digitais (Formas mais elegantes)
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>

//Declarando os pinos dos LEDs
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10

//Declarando os pinos dos Botoes
#define pin_btn1 6
#define pin_btn2 7

int pinos_leds[] = {pin_led1, pin_led2, pin_led3} ;
int pinos_buttons[] = {pin_btn1, pin_btn2} ;

void setup() { // O código dentro do SETUP só executa um vez.

  Serial.begin(9600);

  // Configurando os pinos de saída através do bloco de repetição FOR
  for (int i=0; i <= 2; i++){
    pinMode(pinos_leds[i],OUTPUT);
    Serial.println( (String)"Pino: " + pinos_leds[i] + (String)" configurado como
saída" );
    delay(1000);
  }

  // Configurando os pinos de entrada através do bloco de repetição FOR
  for (int i=0; i <= 1; i++){
    pinMode(pinos_buttons[i],INPUT);
    Serial.println( (String)"Pino: " + pinos_buttons[i] + (String)" configurado como
entrada" );
    delay(1000);
  }
}
```

CONTINUA...



CONTINUAÇÃO

```
void loop() { // O código dentro do LOPP executa infinitamente.

  if (digitalRead(pin_btn1) == 1)
  {
    digitalWrite(pin_led1, HIGH);
  }
  else
  {
    digitalWrite(pin_led1, LOW);
  }

  if (digitalRead(pin_btn2) == 1)
  {
    digitalWrite(pin_led2, HIGH);
    digitalWrite(pin_led3, HIGH);
  }
  else
  {
    digitalWrite(pin_led2, LOW);
    digitalWrite(pin_led3, LOW);
  }

  delay(10);
}
```



ATIVIDADE 1C - TRABALHANDO COM ENTRADAS E SAÍDAS DIGITAIS USANDO FUNÇÕES

Para exemplificar a aplicação de funções, criaremos duas funções. A primeira, chamada de DetectaBotao, será responsável por verificar se determinado botão está acesso, e em caso positivo, acender o LED associado. A segunda função realizará uma soma com dois valores (parâmetros de entrada). Segue o código da atividade:

```
/*
ATIVIDADE 1C - Trabalhando com entradas e saídas digitais (Formas mais elegantes)
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>

//Declarando os pinos dos LEDs
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10

//Declarando os pinos dos Botoes
#define pin_btn1 6
#define pin_btn2 7

int pinos_leds[] = {pin_led1, pin_led2, pin_led3} ;
int pinos_buttons[] = {pin_btn1, pin_btn2} ;

void setup() { // O código dentro do SETUP só executa um vez.
  Serial.begin(9600);

  // Configurando os pinos de saída através do bloco de repetição FOR
  for (int i=0; i <= 2; i++){
    pinMode(pinos_leds[i],OUTPUT);
    Serial.println( (String)"Pino: " + pinos_leds[i] + (String)" configurado como
saída" );
    delay(1000);
  }

  // Configurando os pinos de entrada através do bloco de repetição FOR
  for (int i=0; i <= 1; i++){
    pinMode(pinos_buttons[i],INPUT);
    Serial.println( (String)"Pino: " + pinos_buttons[i] + (String)" configurado como
entrada" );
    delay(1000);
  }
}
```

CONTINUA...



CONTINUAÇÃO

```
//Procedimento para detectar se um botão está pressionado e assim acender um
determinado Led
//Parametros de Entrada: PIN_BTN (Pino do Botao), PIN_LED(Pino do LED)
//Saída: Nenhuma
void DetectaBotao(int pin_btn, int pin_led) {
  if (digitalRead(pin_btn) == 1) {
    digitalWrite(pin_led, HIGH);
  }
  else {
    digitalWrite(pin_led, LOW);
  }
}

delay(15);
}

//Função para somar 2 valores
//Parametros de Entrada: val1, val2
//Saída: float
float PrintSum(double val1, double val2) {
  float result = val1 + val2;
  return result;
}

void loop() { // O código dentro do LOPP executa infinitamente.

  DetectaBotao(pin_btn1, pin_led1);
  DetectaBotao(pin_btn2, pin_led2);
  Serial.println(PrintSum(10,20));

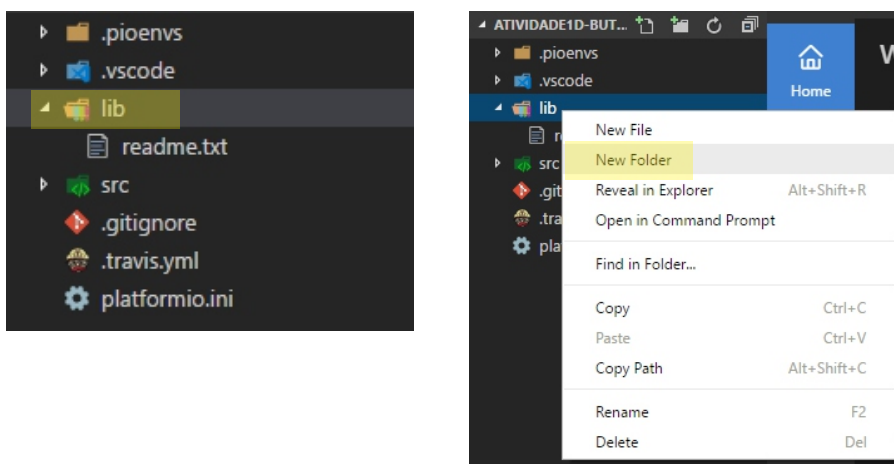
}
```



ATIVIDADE 1 D - TRABALHANDO COM ENTRADAS E SAÍDAS DIGITAIS USANDO BIBLIOTECAS/CLASSES

Para exemplificar o uso das bibliotecas do Arduino, criaremos uma classe chamada LED. Essa classe será responsável por dar “vida” a um simples LED. A classe LED terá características ou propriedades que serão responsáveis pelo comportamento dos objetos do tipo Led. As classes são muito utilizadas nas implementações de bibliotecas de dispositivos que se conectam aos microcontroladores. São elas que tornam a vida do programador mais simples e prática, pois elas escondem ou abstraem grande parte do “serviço sujo” que teríamos que fazer para que um determinado Sensor , ou Display, funcionasse corretamente.

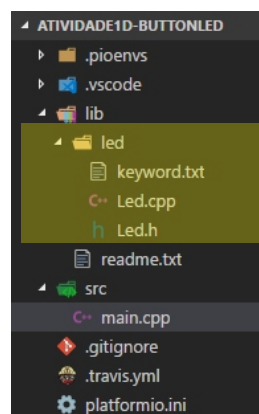
Para criar uma biblioteca no Visual Code, precisaremos criar um pasta chamada “led”, dentro da pasta “lib” presente no diretório do nosso projeto. Para isso, basta clicar com o botão direito do mouse na pasta “lib” e clicar na opção “New Folder”.



Em seguida, repita o procedimento já na pasta “led” e crie dentro dela três arquivos.

- 1- Led.cpp
- 2- Led.h
- 3- keywords.txt

Assim a estrutura do seu projeto deverá ficar desta forma:





Led.h - Nosso HEADER!

Os arquivos terminados em ".h" são conhecidos como *headers*. Em nossa atividade, no arquivo "Led.h", será criada a classe Led, bem como, "descritas" todas as suas propriedades e funções. Repare no código a seguir, que temos a estrutura de declaração da classe, algumas variáveis e apenas os cabeçalhos das funções "ligar", "desligar" e "piscar". Pois bem, esta é a função dos *headers*. Apenas descrever quais propriedades e funções a determinada classe possuirá, não se encarregando de implementar tais funções ou procedimentos.

```
#include <Arduino.h>

//DECLARAÇÃO DA CLASSE LED
class Led {
public: //TUDO QUE ESTIVER NO PUBLIC É VISTO
    Led(int pin); // CONSTRUCTOR DA CLASSE LED
    int pino; // Propriedade Pino do LED
    String cor; // Propriedade Cor do LED
    String descricao; //Propriedade Descrição do LED
    void ligar(); //Função para ligar o led
    void desligar(); //Função para desliga o led
    void piscar(int tempo); //Função para piscar o led com intervalo definido
    (tempo (ms))
};
```

Led.cpp - Nosso implementador!

Nos arquivos terminados em ".cpp" é que iremos implementar as funções e procedimentos descritos nos *headers*. Assim, segue o conteúdo da implementação das funções na nossa classe Led.

```
#include <Arduino.h>
#include "Led.h" // Precisamos incluir o arquivo header no contexto.

//Implementação do Construtor da classe.
Led::Led(int pin){
    pino = pin;
    pinMode(pin,OUTPUT);
};

//Implementação da função ligar.
void Led::ligar(){
    digitalWrite(pino,HIGH);
};

//Implementação da função desligar.
void Led::desligar(){
    digitalWrite(pino,LOW);
};

//Implementação da função piscar.
void Led::piscar(int tempo){
    digitalWrite(pino,HIGH);
    delay(tempo);
    digitalWrite(pino,LOW);
    delay(tempo);
};
```



keyword.txt - Nosso mapa de destaques!

Os arquivos *keyword.txt* servem para destacarmos palavras chaves da nossa classe durante a implementação dos códigos. Você já deve ter reparado que determinadas palavras tanto na IDE do Arduino, quanto no PlatformIO, possuem coloração diferente. Pois é, quem determina quais palavras serão destacadas, é o conteúdo do arquivo *keyword.txt*. No nosso caso, o *keyword.txt* deverá ter o seguinte conteúdo.

```
Led          KEYWORD1
ligar        KEYWORD2
desligar     KEYWORD2
```

main.cpp - Nosso programa de fato!

Para usarmos nossa classe *Led*, instanciaremos o objeto *Led* em nosso programa e assim faremos uso das funcionalidades atribuídas a classe. Segue o código:

```
/*
ATIVIDADE 1D - Trabalhando com entradas e saídas digitais (Usando
Classes/Bibliotecas/Libraries)
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>
#include <Led.h> // Indica os arquivos da classe LED (bibliotecas)

Led Led1(8); // Instancia o objeto Led1 da Classe Led
Led Led2(9); // Instancia o objeto Led2 da Classe Led
Led Led3(10); // Instancia o objeto Led3 da Classe Led

void setup() {
  Serial.begin(9600);
  Led1.descricao = "Led número 1"; // Define a propriedade descricao do LED1 (String)
  Led1.cor = "Vermelho" ; // Define a propriedade cor do LED1 (String)
  Led2.descricao = "Led número 2"; // Define a propriedade descricao do LED1 (String)
  Led2.cor = "Azul" ; // Define a propriedade cor do LED1 (String)
  Led3.descricao = "Led número 3"; // Define a propriedade descricao do LED3 (String)
  Led3.cor = "Verde" ; // Define a propriedade cor do LED3 (String)
}
```

CONTINUA...



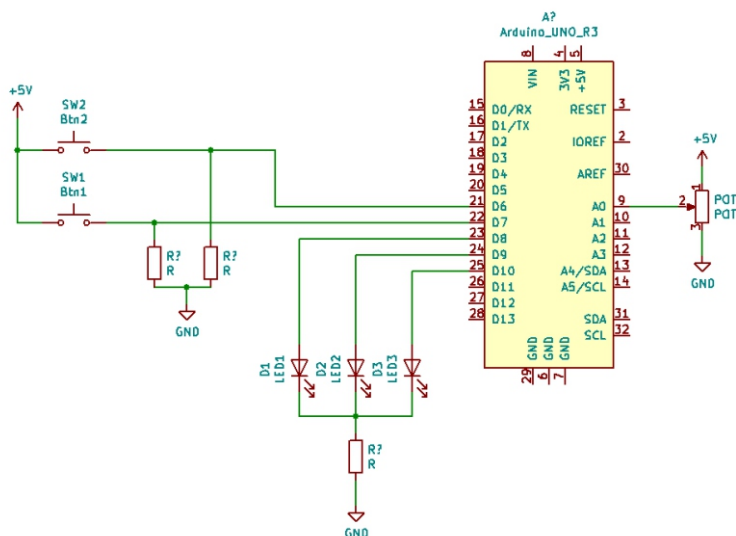
CONTINUAÇÃO

```
void loop() {  
  // Imprime na serial a propriedade descricao do LED1,número do pino e sua cor  
  Serial.println(String(Led1.descricao) + " : " + String(Led1.pino) + " : " +  
  String(Led1.cor));  
  // Imprime na serial a propriedade descricao do LED2,número do pino e sua cor  
  Serial.println(String(Led2.descricao) + " : " + String(Led1.pino) + " : " +  
  String(Led2.cor));  
  // Imprime na serial a propriedade descricao do LED3,número do pino e sua cor  
  Serial.println(String(Led3.descricao) + " : " + String(Led3.pino) + " : " +  
  String(Led3.cor));  
  
  Led1.ligar(); // Liga o Led1  
  delay(2000); // Aguarda  
  Led1.desligar(); // Desliga o Led1  
  delay(2000); // Aguarda  
  
  Led2.piscar(100); // Pisca o LED2  
  Led3.piscar(1000); //Piscar o LED3  
  
  }cor = "Verde" ; // Define a propriedade cor do LED3 (String)  
}
```



ATIVIDADE 2A - TRABALHANDO COM ENTRADAS ANALÓGICAS E PROTOCOLO SERIAL

Neste momento, após estudarmos algumas técnicas de programação, iremos trabalhar com entradas analógicas. Para isto, deve-se montar na protoboard o esquemático abaixo. Repare que em relação as atividades anteriores, foi apenas adicionado um potenciômetro, que irá simular nosso sensor analógico.



Nesta atividade, o potenciômetro irá simular um sensor de temperatura analógico. O conversor AD do Arduino possui resolução de 10bits. Ou seja, ele converte as tensões de 0 a 5V em valores inteiros de 0 a 1023. Assim, vamos supor que na temperatura de -100°C, o sensor apresente 2.5V na sua saída. Quando atinge +100°C, o sensor irá apresentar 5V em sua saída.

Utilizando a interface serial, vamos criar um protocolo onde iremos solicitar através de uma "string", os valores de temperatura, bem como, o status dos botões. Através do nosso protocolo, vamos também, acender e apagar os leds.

O protocolo que será criado terá o seguinte formato para as strings de solicitação:

INI	CMD	PAR	FIM	*
-----	-----	-----	-----	---

INI - CARACTER DE INÍCIO " ("

* - SEPARADOR DE REQUISIÇÕES

CMD - COMANDO QUE DESEJA SE ENVIAR AO ARDUINO

PAR - PARÂMETROS DE ENTRADA DO COMANDO

FIM - CARACTER DE FINAL ") "

CMD	Parâmetros	Descrição
LERBTN	1 - Botão 1 2 - Botão 2	Lê o status do botão 1 ou 2
LERTEMP	SEM PARÂMETROS	Lê o valor de temperatura do termômetro
LEDON	1 - LED 1 2 - LED 2	Liga o Led 1 ou Led 2
LEDOFF	1 - LED 1 2 - LED 2	Desliga o Led 1 ou Led 2



```
/*
ATIVIDADE 2A - Trabalhando com entradas analógicas e protocolo serial
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>

unsigned long t1;
unsigned long t2;

//Declarando os pinos dos LEDs
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10

//Declarando os pinos dos Botoes
#define pin_btn1 6
#define pin_btn2 7

//Declarando o pino do Potenciometro (Sensor)
#define pin_sensor1 A0

int pinos_leds[] = {pin_led1, pin_led2, pin_led3};
int pinos_buttons[] = {pin_btn1, pin_btn2};

//Variáveis para tratamento do protocolo
int indice[4] = {0, 0, 0, 0};
String cmd;
String parametro;
int char_pos = 0;
String LinhaDados = "";

void setup()
{ // O código dentro do SETUP só executa um vez.

  Serial.begin(115200); // Configura a interface serial
  // Configurando os pinos de saída através do bloco de repetição FOR
  for (int i = 0; i <= 2; i++)
  {
    pinMode(pinos_leds[i], OUTPUT);
    Serial.println((String) "Pino: " + pinos_leds[i] + (String) " configurado como
saída");
    delay(50);
  }
  // Configurando os pinos de entrada através do bloco de repetição FOR
  for (int i = 0; i <= 1; i++)
  {
    pinMode(pinos_buttons[i], INPUT);
    Serial.println((String) "Pino: " + pinos_buttons[i] + (String) " configurado
como entrada");
    delay(50);
  }
}
```

CONTINUA...



CONTINUAÇÃO

```
void LerBtn(int btn)
{
    switch (btn)
    {
        case 1:
            if (digitalRead(pinos_buttons[0]) == 1)
            {
                Serial.println(String("Botao 1 - Pressionado"));
            }
            else
            {
                Serial.println(String("Botao 1 - Nao pressionado"));
            }
            break;
        case 2:
            if (digitalRead(pinos_buttons[1]) == 1)
            {
                Serial.println(String("Botao 2 - Pressionado"));
            }
            else
            {
                Serial.println(String("Botao 2 - Nao pressionado"));
            }
            break;
        default:
            break;
    }
}

float tempCelsius(int val_digital)
{
    Serial.println(analogRead(pin_sensor1));
    float tensao = val_digital * (5.0 / 1023.0);
    Serial.println(tensao);
    float temperatura = (200.0 / 5.0) * tensao - 100.0;
    return temperatura;
}

void LerTemp()
{
    Serial.println(String("Temperatura: ") + tempCelsius(analogRead(pin_sensor1)) +
String("C"));
}

void LigarLED(int led)
{
    switch (led)
    {
        case 1:
            digitalWrite(pinos_leds[0], HIGH);
            break;
        case 2:
            digitalWrite(pinos_leds[1], HIGH);
            break;
        case 3:
            digitalWrite(pinos_leds[2], HIGH);
            break;
        default:
            break;
    }
}
```

CONTINUA...



CONTINUAÇÃO

```
void DesligarLED(int led)
{
    switch (led)
    {
        case 1:
            digitalWrite(pinos_leds[0], LOW);
            break;
        case 2:
            digitalWrite(pinos_leds[1], LOW);
            break;
        case 3:
            digitalWrite(pinos_leds[2], LOW);
            break;
        default:
            break;
    }
}

void loop()
{
    if (Serial.available())
    {
        LinhaDados = Serial.readStringUntil('*');

        if ((LinhaDados.charAt(0) == '(') && ((LinhaDados.charAt(LinhaDados.length() - 1)) == ')')) //String do protocolo recebido
        {
            t1 = micros();
            //Serial.println((String("Mensagem Recebida:")) + LinhaDados);
            indice[0] = LinhaDados.indexOf(','); // Encontra a posição da virgula (,)
            cmd = LinhaDados.substring(1, indice[0]); //Copia a substring da posicao 1
            até antes da virgula
            indice[1] = LinhaDados.indexOf(')', indice[0] + 1); // Encontra a posição
            do caracter F
            parametro = LinhaDados.substring(indice[0] + 1, indice[1]); //Copia
            substring da posicao da próxima da virgula até o F
            //Serial.println(String("Comando: ") + cmd);
            //Serial.println(String("Parametro: ") + parametro);
            if (cmd == "LERBTN") //Verificação dos comandos
            {
                LerBtn(parametro.toInt());
            }
            if (cmd == "LERTEMP")
            {
                LerTemp();
            }
            if (cmd == "LEDON")
            {
                LigarLED(parametro.toInt());
            }
            if (cmd == "LEDOFF")
            {
                DesligarLED(parametro.toInt());
            }
        }
        else
        {
            Serial.println("ERRO");
        }
    }
}
```

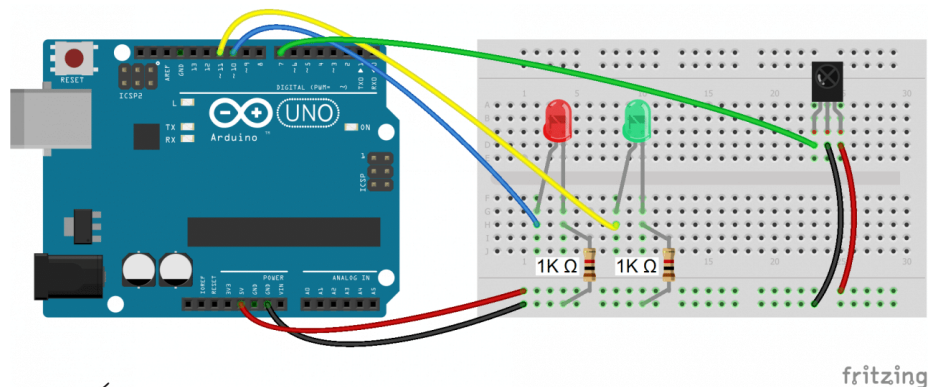


ATIVIDADE 2B - TRABALHANDO SENSOR INFRAVERMELHO

Nesta atividade utilizaremos o sensor IR para detectar comandos de um controle remoto. Mesmo ainda não trabalhando conectados com a internet, essa aplicação vai dar suporte para criarmos aplicações interessantes no futuro para Internet das Coisas. Segue o esquemático eletrônico:

MAPA DE TECLAS CONTROLE REMOTO - HEXADECIMAL

CH-	FFA25D
CH	FF629D
CH+	FFE21D
PREV	FF22DD
NEXT	FF02FD
PLAY	FFC23D
VOL-	FFE01F
VOL+	FFA857
EQ	FF906F
0	Ff6897
100+	Ff9867
200+	FFB04F
1	FF30CF
2	FF18E7
3	FF7A85
4	FF10EF
5	FF38C7
6	FF5AA5
7	FF42BD
8	FF4AB5
9	FF52AD



CÓDIGO FONTE

```
/*
ATIVIDADE 2B - Controlando o Arduino via Controle Remoto IR
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>
#include <IRremote.h>

#define PinoReceptorIR 7 //Define o pino do sensor IR no Arduino

IRrecv ReceptorIR(PinoReceptorIR); //Criando o objeto da biblioteca IRRemote
decode_results SinalIRDecodificado; // Criando o objeto da biblioteca IRRemote para
o códigos decodificados do Sensor IR

void setup() {

    pinMode(13, OUTPUT); // Define o pino 13 como saída
    Serial.begin(9600); //Inicializa a serial em 9600 de BaudRate
    ReceptorIR.enableIRIn(); // Ativa o receptor IR

}
```

CONTINUA...



CONTINUAÇÃO

```
void loop() {  
    if (ReceptorIR.decode(&SinalIRDecodificado)) //Se o ReceptorIR receber o código de  
    algum botão do controle remote: Salva no objeto SinalIRDecodificado  
    {  
        digitalWrite(13,HIGH); // Acende o Led do Arduino  
        Serial.print("CódigoIR: ");  
        Serial.print(SinalIRDecodificado.value, HEX); //Imprime o valor em hexadecimal  
        Serial.print(", bits: ");  
        Serial.println(SinalIRDecodificado.bits); //Imprime o valor em binário  
        ReceptorIR.resume(); // Receive the next value  
    }  
    delay(600); //delay 600ms  
    digitalWrite(13,LOW);  
}
```



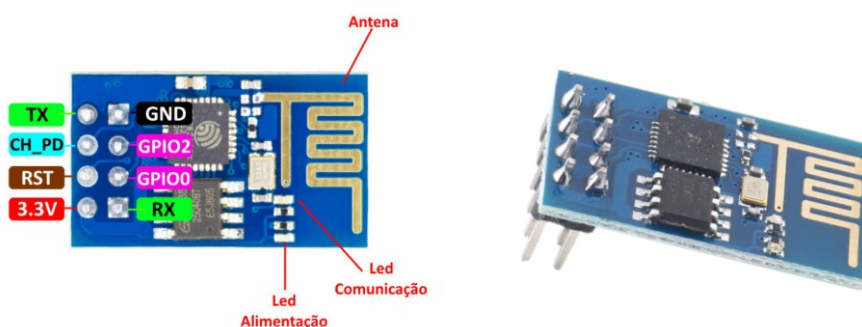
QUEM É ESP8266?

O ESP8266 é um circuito integrado SOC que possui uma pilha de protocolo TCP/IP integrada que pode fornecer a qualquer microcontrolador, acesso à sua rede Wi-Fi. O ESP8266 é capaz de hospedar um aplicativo ou descarregar todas as funções de rede Wi-Fi providas de outro dispositivo, como o Arduino. Cada módulo ESP8266 vem pré-programado com um firmware contendo um conjunto de comandos AT, o que significa que você pode simplesmente conectá-lo ao seu dispositivo Arduino e obter as funcionalidades Wi-Fi. O módulo ESP8266 é uma placa extremamente barata e com uma enorme e crescente comunidade.

CARACTERÍSTICAS

- Wi-Fi embutido: padrões 802.11 b/g/n
- Alcance aproximado (área livre): 90 metros
- Tensão de operação: 3.3 Volts
- Pilha do protocolo TCP/IP integrado
- CPU que opera em 80 Mhz, com possibilidade de OC de 160 Mhz
- Arquitetura RISC de 32 bits;
- 32KBytes de RAM para instruções;
- 96KBytes de RAM para dados;
- 64KBytes de ROM para boot;
- Possui uma memória Flash SPI Winbond W25Q40BVNIG de 512KBytes a 4 Mb * (suporte até 16 Mb)
- O núcleo é baseado no IP Diamond Standard LX3 da Tensilica;
- Modos de operação: Cliente, Access Point, Cliente+Access Point
- Fabricado pela Espressif;
- Existem módulos de diferentes tamanhos e fabricantes.

PINAGEM





TIPOS DE ESP8266

ESP01



ESP02



ESP03



ESP04



ESP05



ESP06



ESP07



ESP08



ESP09



ESP10



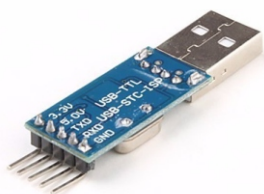
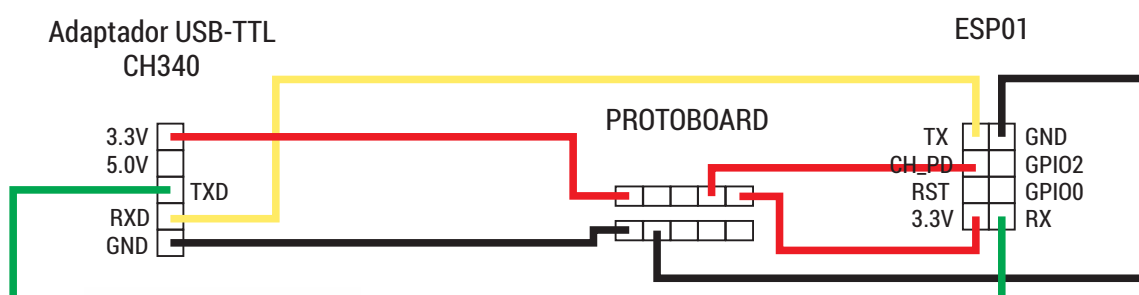
ESP11



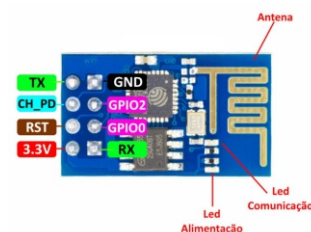
ESP12



CONECTANDO O ESP01 COM O PC ATRAVÉS DO ADAPTADOR USB



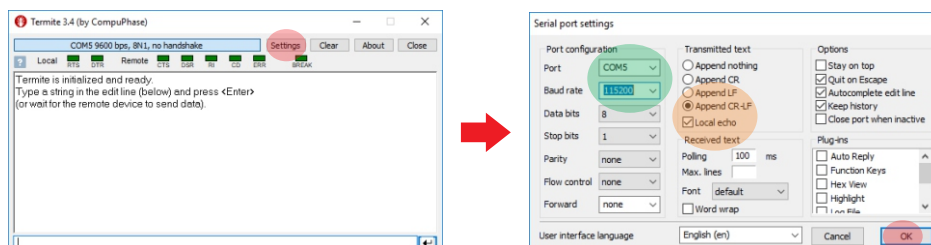
DEIXAR OS PINOS GPIO2, GPIO0 e RST COM CABOS, PORÉM SEM CONEXÃO



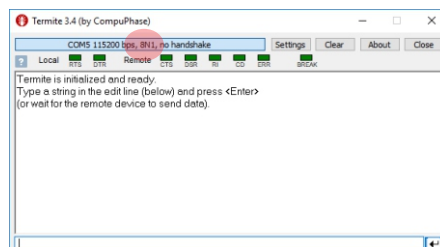


ESP8266 - O queridinho da IoT

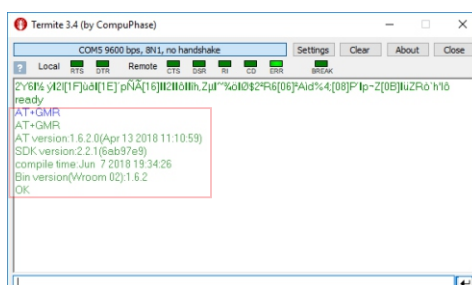
Após concluir as ligações elétricas entre o Adaptador USB e o módulo ESP01, devemos abrir um terminal de conexões serial. Recomendo o uso do aplicativo Ttermite. Conecte o adaptador USB no PC e configure o Ttermite para realizar a conexão com o *baudrate* de 115200. Marque a opção Append CR-LF e Local Echo.



Após configurar o terminal, clique sobre a porta COM, e a conexão será aberta.



Neste momento, o terminal deve começar a receber algumas informações do módulo ESP01. Digite o comando "AT+GMR". Informações do *firmware* instalado no ESP01 deve ser mostrado terminal.



AJUSTANDO O BAUDRATE

Antes de iniciarmos nossa integração entre Arduino e ESP01, precisaremos configurar o *baudrate* do ESP01 para trabalharem em 9600 bps. Isso porquê, em *baudrates* maiores, o Arduino tem dificuldade em se comunicar utilizando a biblioteca Softserial. Sendo assim, com o terminal conectado, digite o comando "AT+UART_DEF=9600,8,1,0,0". Se tudo ocorrer bem, a mensagem de OK deve ser retornada pelo ESP01.

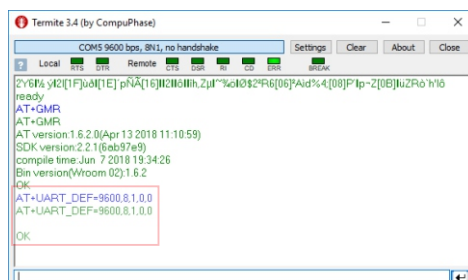




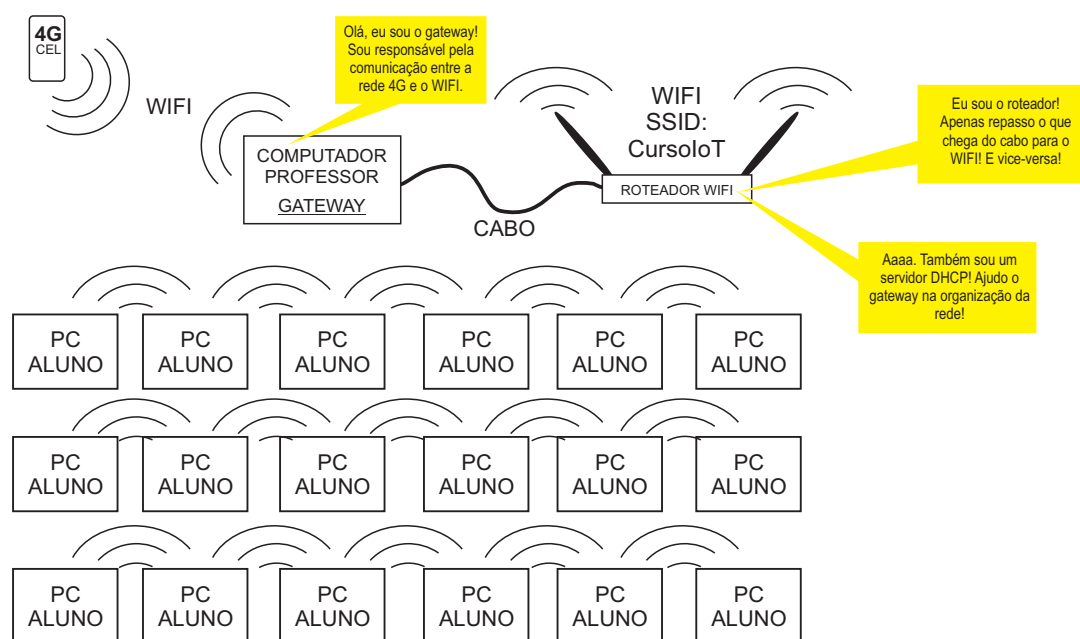
TABELA DE COMANDOS AT

Function	AT Command	Response
Working	AT	OK
Restart	AT+RST	OK [System Ready, Vendor:www.ai-thinker.com]
Firmware version	AT+GMR	AT+GMR 0018000902 OK
List Access Points	AT+CWLAP	AT+CWLAP +CWLAP:(4,"RocheFortSurLac",-38,"70:62:b8:6f:6d:58",1) +CWLAP:(4,"LiliPad2.4",-83,"f8:7b:8c:1e:7c:6d",1) OK
Join Access Point	AT+CWJAP? AT+CWJAP="SSID","Password"	Query AT+CWJAP? +CWJAP:"RocheFortSurLac" OK
Quit Access Point	AT+CWQAP=? AT+CWQAP	Query OK
Get IP Address	AT+CIFSR	AT+CIFSR 192.168.0.105 OK
Set Parameters of Access Point	AT+ CWSAP? AT+ CWSAP= <ssid>,<pwd>,<chl>,<ecn>	Query ssid, pwd chl = channel, ecn = encryption
WiFi Mode	AT+CWMODE? AT+CWMODE=1 AT+CWMODE=2 AT+CWMODE=3	Query STA AP BOTH
Set up TCP or UDP connection	AT+CIPSTART=? (CIPMUX=0) AT+CIPSTART = <type>,<addr>,<port> (CIPMUX=1) AT+CIPSTART= <id><type>,<addr>,<port>	Query id = 0-4, type = TCP/UDP, addr = IP address, port= port
TCP/UDP Connections	AT+ CIPMUX? AT+ CIPMUX=0 AT+ CIPMUX=1	Query Single Multiple
Check join devices' IP	AT+CWLIF	
TCP/IP Connection Status	AT+CIPSTATUS	AT+CIPSTATUS? no this fun
Send TCP/IP data	(CIPMUX=0) AT+CIPSEND=<length>; (CIPMUX=1) AT+CIPSEND= <id>,<length>	
Close TCP / UDP connection	AT+CIPCLOSE=<id> or AT+CIPCLOSE	
Set as server	AT+ CIPSERVER= <mode>[,<port>]	mode 0 to close server mode; mode 1 to open; port = port
Set the server timeout	AT+CIPSTO? AT+CIPSTO=<time>	Query <time>0~28800 in seconds
Baud Rate*	AT+CIOBAUD? Supported: 9600, 19200, 38400, 74880, 115200, 230400, 460800, 921600	Query AT+CIOBAUD? +CIOBAUD:9600 OK
Check IP address	AT+CIFSR	AT+CIFSR 192.168.0.106 OK
Firmware Upgrade (from Cloud)	AT+CIUPDATE	1. +CIPUPDATE:1 found server 2. +CIPUPDATE:2 connect server 3. +CIPUPDATE:3 got edition 4. +CIPUPDATE:4 start update
Received data	+IPD	(CIPMUX=0): + IPD, <len>: (CIPMUX=1): + IPD, <id>,<len>: <data>
Watchdog Enable*	AT+CSYSWDTENABLE	Watchdog, auto restart when program errors occur: enable
Watchdog Disable*	AT+CSYSWDTDISABLE	Watchdog, auto restart when program errors occur: disable



COMO ESTAMOS INTERLIGADOS COM O MUNDO?

Para realizar as atividades do curso, a infra-estrutura de rede foi configurada da seguinte forma:



O Gateway é o dispositivo responsável por interligar redes ou subredes distintas. Redes distintas não necessariamente precisam estar em meios físicos diferentes. No nosso caso, a rede 4G, atribui um endereço IP para o celular, chamamos esse endereço de IP real.

IP real é um endereço lógico na faixa de endereço visíveis na internet. Assim, todo dispositivo que possui um IP real pode ser acessado de qualquer lugar do mundo, através da internet.

O celular também faz um papel de gateway, permitindo que o computador do professor, se comunique com a internet através de um novo IP (pelo sinal Wifi criado entre PC-Celular). Agora, esse IP não é chamado de IP real. Neste caso, esse endereço passa ser um IP local. Dessa forma, não é possível acessar diretamente o computador do professor, apenas com o IP local que o celular atribuiu a ele pelo Wifi. O IP local trabalha em outra faixa de endereços.

Além disso, o PC do professor passa ser um Gateway também. Ele interliga a interface da rede cabeada que opera numa faixa de IPs locais, com a rede Wifi que ele mantém com o celular 4G. Por fim, o roteador apenas interliga dois meios diferentes, o cabeado que vem do PC com a rede Wifi que ele criou para permitir que os alunos acessem a internet durante o curso.

O roteador também exerce a função de servidor DHCP. É ele quem gera os endereços locais para os computadores dos alunos e distribui via Wifi.



PROTOCOLO TCP/IP

O TCP/IP é um conjunto de protocolos de comunicação entre computadores em rede (também chamado de Pilha de Protocolos TCP/IP). Seu nome vem de dois principais protocolos utilizados:

- TCP (Transmission Control Protocol – Protocolo de Controle de Transmissão)
- IP (Internet Protocol – Protocolo de Interconexão).
-

O conjunto de protocolos pode ser visto como um modelo de camadas, onde cada camada é responsável por um grupo de tarefas, fornecendo um conjunto de serviços bem definidos para o protocolo da camada superior. As camadas mais altas estão logicamente mais perto do usuário (chamada camada de aplicação) e lidam com dados mais abstratos, confiando em protocolos de camadas mais baixas para tarefas de menor nível de abstração.

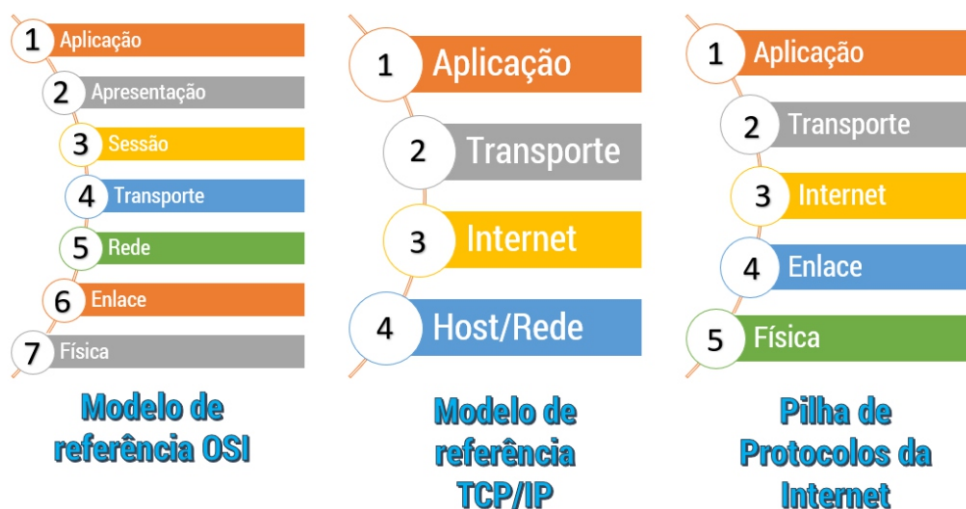
Conforme DIOGENES (2004), os principais objetivos da criação do Protocolo TCP/IP foram:

- Obter um protocolo que fosse compatível com todos os tipos de redes;
- Que fosse interoperável entre todos os fabricantes;
- Possuísse uma comunicação robusta (confiável e com baixo índice de falhas), escalonável (passível de ser colocada em níveis ou etapas) e que suportasse o crescimento das redes de uma forma segura e confiável;
- E que fosse dinâmico e de fácil configuração.

O protocolo surgiu por volta de 1960, desenvolvido pelo Departamento de Defesa Americano, com o intuito de preservar a integridade dos dados, sem que os mesmos fossem interceptados por inimigos (principalmente em épocas de guerra) (FILIPPETTI, 2002).

PILHAS DE PROTOCOLO DA INTERNET

A internet utiliza uma pilha de protocolos resultado da mistura das duas pilhas de protocolos (OSI e TCP/IP). O resultado desta mistura, gerou um Modelo de abstração em 5 camadas:





- Camada de Aplicação: Suporta as aplicações da rede. Ex.: FTP, SMTP, HTTP...
- Camada de Transporte: Transferência de dados, sistema final a sistema final. Ex.: TCP e UDP
- Camada de Rede: Roteamento de datagramas da origem ao destino. Ex.: IP, protocolos de roteamento.
- Camada de Enlace: Transferência de dados entre elementos vizinhos da rede. Ex.: PPP, Ethernet...
- Camada Física: Bits no meio de transmissão. Ex.: Pulsos elétricos no Cabo UTP.

MODELO OSI vs MODELO TCP/IP

Semelhanças:

- Ambos têm camadas;
- Ambos têm camadas de aplicação, embora incluam serviços muito diferentes;
- Ambos têm camadas de transporte e de rede comparáveis;
- A tecnologia de comutação de pacotes (e não comutação de circuitos) é presumida por ambos;
- Os profissionais da rede precisam conhecer ambos.

Diferenças:

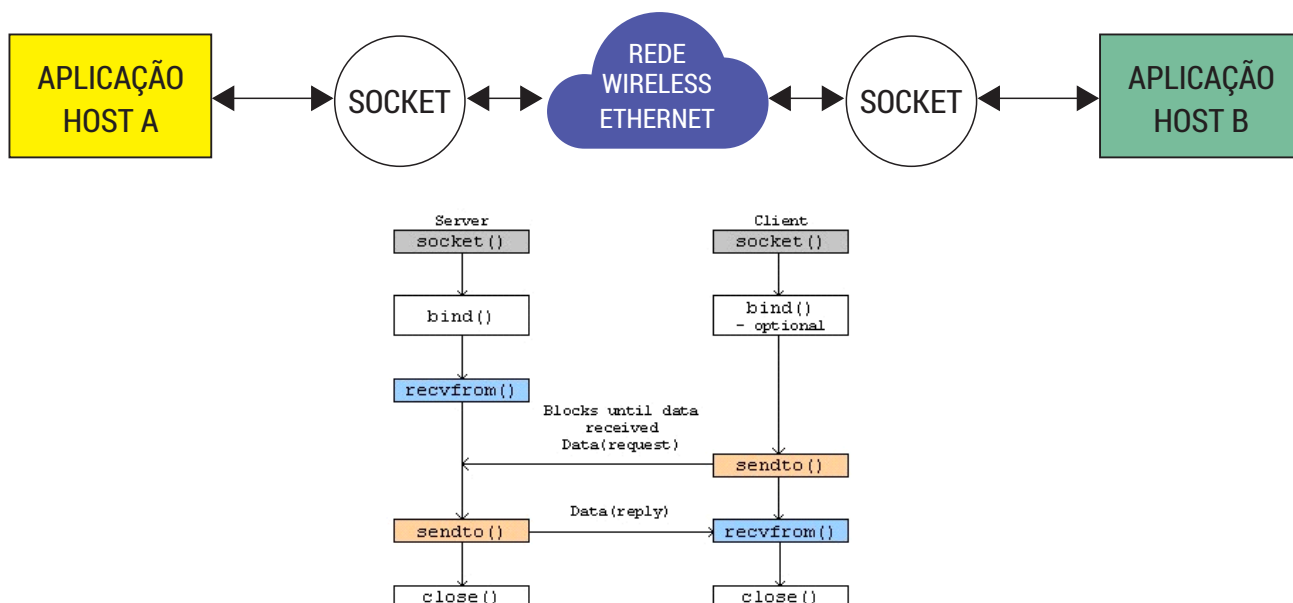
- TCP/IP combina os aspectos das camadas de apresentação e de sessão dentro da sua camada de aplicação;
- TCP/IP combina as camadas física e de enlace do OSI em uma camada;
- TCP/IP parece ser mais simples por ter menos camadas;
- Os protocolos do TCP/IP são os padrões em torno dos quais a Internet se desenvolveu, portanto o modelo TCP/IP ganha credibilidade apenas por causa dos seus protocolos. Em contraste, nenhuma rede foi criada em torno de protocolos específicos relacionados ao OSI, embora todos usem o modelo OSI para guiar seu raciocínio.

LEITURA
RECOMENDADA!

Você sabe o que é o Modelo OSI?

<https://www.uniaogeek.com.br/voce-sabe-o-que-e-o-modelo-osi/>

SOCKETS - CAMADA DE APLICAÇÃO

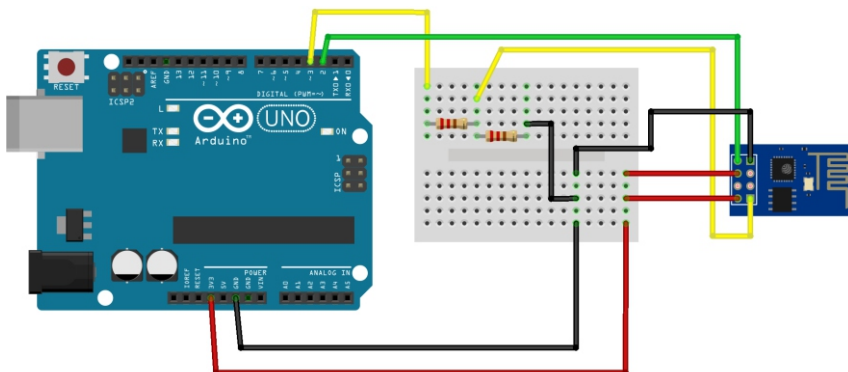




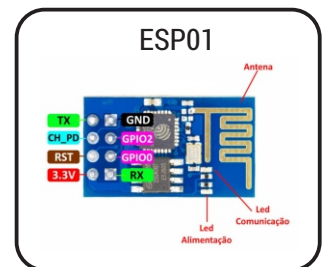
ATIVIDADE 3A - PRIMEIRO CONTATO COM O ESP8266

Nesta atividade, iremos conectar eletronicamente o ESP01 com o Arduino. Essa aplicação será bem simples, o intuito é averiguar se a comunicação entre eles está funcionando adequadamente, já que o Arduino deverá controlar o ESP01 através dos comandos AT.

Primeiramente, faça as ligações entre o Arduino e o ESP01 de acordo com esquema abaixo.



fritzing



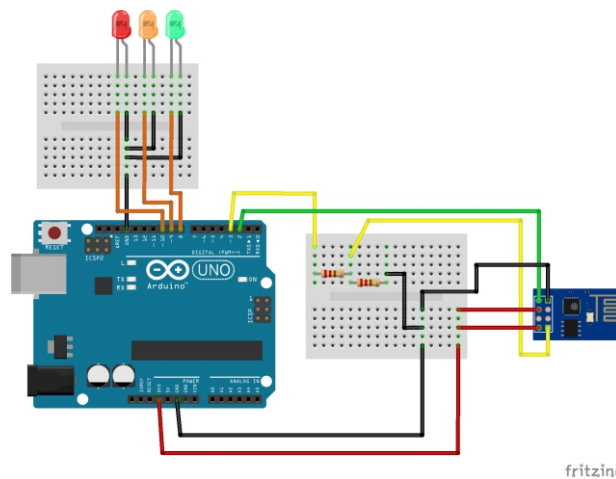
CÓDIGO FONTE

```
/*
ATIVIDADE 3A - Comunicação Transparente entre Arduino e ESP01
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/
#include <Arduino.h>
#include <SoftwareSerial.h>
//Cria uma nova interface serial através dos Pinos D2 e D3 do Arduino,
//onde será conectado o módulo wifi ESP-01
// Pino 2 é o RX do Arduino (TX do ESP)
// Pino 3 é o TX do Arduino (RX do ESP) - Precisa do divisor de tensão 5V -> 3.3V
SoftwareSerial ESP01(2, 3);
void setup()
{
  Serial.begin(9600); // Inicia a comunicação entre o PC e o Arduino
  ESP01.begin(9600); // Inicia a comunicação entre Arduino e o ESP-01
  Serial.println("Comunicação PC-Arduino-ESP01");
  Serial.println("Lembre-se de setar 'Ambos, NL e CR' no serial monitor.");
  Serial.println("Ready");
}
void loop(){
  // Escuta a comunicação serial do ESP-01 e escreve na serial do Arduino
  if (ESP01.available()) {
    Serial.write(ESP01.read());
  }
  // Escuta a comunicação serial do Arduino e escreve na serial do ESP-01
  if (Serial.available()) {
    ESP01.write(Serial.read());
  }
}
```



ATIVIDADE 3B - ACENDENDO LEDS PELO WIFI

Essa atividade tem por objetivo ligar e desligar leds através da comunicação Wi-Fi, utilizando o protocolo TCP/IP via sockets. Para muitos, essa parte do curso se torna nova, e uma explanação básica sobre protocolos de internet será feita pelo professor. Dessa forma, você irá compreender o que está acontecendo por dentro do Wi-Fi. Para iniciarmos, monte o esquemático abaixo, adicionando os leds nos pinos 8,9 e 10. Observe que a ligação entre o Arduino e ESP01 continua idêntica à atividade anterior.



Nessa configuração, o ESP-01 será responsável pela conexão com a rede Wi-Fi e o Arduino ficará no aguardo de comandos via serial para controlar os leds. O ESP-01 será comunicar com o Arduino através dos comandos AT. Quando um comando chegar via Socket/Wi-Fi no ESP-01, ele repassará tal comando para o Arduino via serial.

CÓDIGO FONTE

```
/*
ATIVIDADE 3BA - ServerSocket para acionamento de Leds/Relés remotamente.
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>
#include <SoftwareSerial.h> //Biblioteca para criação de portas seriais (softserial)

//Declarando os pinos dos LEDs
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10
int pinos_leds[] = {pin_led1, pin_led2, pin_led3};

//Cria uma nova interface serial através dos Pinos D2 e D3 do Arduino,
//onde será conectado o módulo wifi ESP-01
// Pino 2 é o RX do Arduino (TX do ESP)
// Pino 3 é o TX do Arduino (RX do ESP) - Precisa do divisor de tensão 5V -> 3.3V
SoftwareSerial ESP01(2, 3);
```

CONTINUA...



CONTINUAÇÃO

```
boolean EnviaComandoESP01(String cmd, String ack, unsigned timeout, String descricao,
String retorno){
// Função para envio de comandos ATs para o ESP-01
// Parametros de Entrada: Cmd - Comando , ACK - retorno
// Parametros de Saída: Verdadeiro ou Falso (true or false)
Serial.print(descricao);
ESP01.println(cmd); // Envia o comando (cmd) para o ESP01
timeout = timeout + millis();
while(millis() < timeout){
//Serial.println(cmd + ":" + "Esperando ack...");
if (ESP01.available()){
String resposta = ESP01.readString();
resposta.trim();
if (resposta.indexOf(ack) != -1){

    if (ack == "+CIFSR:STAIP,") {
        int i1 = resposta.indexOf("+CIFSR:STAIP,") + 13; //13 é o tamanho de
"+CIFSR:STAIP,"
        int i2 = resposta.indexOf("+", i1+2);
        Serial.print("IP:");
        String ip = resposta.substring(i1,i2);
        ip.trim();
        Serial.println(ip);
        return true;
    }
    Serial.println(retorno);
    return true;
}
}
}
return false;
}

void setup()
{
pinMode(pinos_leds[0],OUTPUT);
pinMode(pinos_leds[1],OUTPUT);
pinMode(pinos_leds[2],OUTPUT);
Serial.begin(9600);
ESP01.begin(9600);

EnviaComandoESP01("AT+RST", "Ready",5000,"ESP01 - Resetando o ESP01", "Reset OK");
EnviaComandoESP01("AT+CWQAP", "OK", 5000,"ESP01 - Desconectando de Redes Anteriores...",
"Desconectado!");
EnviaComandoESP01("AT+CWMODE=1", "OK",5000,"ESP01 - Setando Modo Station...", "Setado!");
EnviaComandoESP01("AT+CWJAP=\"CursoIoT\", \"julho2018\"", "WIFI GOT IP",20000,"ESP01 -
Conectando na Rede CursoIoT...", "Conectado!");
EnviaComandoESP01("AT+CIFSR", "+CIFSR:STAIP,",5000,"ESP01 - Adquirindo numero de
IP...", "IP Adquirido:");
EnviaComandoESP01("AT+CIPMUX=1", "OK",5000,"ESP01 - Setando TCPServer para multiplas
conexoes (Max 5) ...", "TCPServer Multiplicas Conexoes OK");
EnviaComandoESP01("AT+CIPSERVER=1,80", "OK",5000,"ESP01 - Startando o TCPServer na porta
80...", "TCPServer Operante");
}
```

CONTINUA...



CONTINUAÇÃO

```
void loop(){

String StringEntrada=""; //Armazena as Strings vindas do Wifi
boolean StringPronta = false; // Flag de controle das strings vindas do Wifi

while (ESP01.available()){
    StringEntrada=ESP01.readString();
    StringEntrada.trim(); // Remove caracteres fantasmas (single space (ASCII 32), tab
(ASCII 9), vertical tab (ASCII 11), form feed (ASCII 12), carriage return (ASCII 13),
or newline (ASCII 10))
    StringPronta = true;
}

if (StringPronta){

    if (StringEntrada.indexOf("LED1=ON") != -1) { //Se encontrar a String "LED1=ON"
dentro da StringEntrada
        digitalWrite(pinos_leds[0],HIGH);
        Serial.println("LED1 - Aceso" );
    }

    if (StringEntrada.indexOf("LED2=ON") != -1) {
        digitalWrite(pinos_leds[1],HIGH);
        Serial.println("LED2 - Aceso" );
    }

    if (StringEntrada.indexOf("LED3=ON") != -1) {
        digitalWrite(pinos_leds[2],HIGH);
        Serial.println("LED3 - Aceso" );
    }

    if (StringEntrada.indexOf("LED1=OFF") != -1) {
        digitalWrite(pinos_leds[0],LOW);
        Serial.println("LED1 - Apagado" );
    }

    if (StringEntrada.indexOf("LED2=OFF") != -1) {
        digitalWrite(pinos_leds[1],LOW);
        Serial.println("LED2 - Apagado" );
    }

    if (StringEntrada.indexOf("LED3=OFF") != -1) {
        digitalWrite(pinos_leds[2],LOW);
        Serial.println("LED3 - Apagado" );
    }

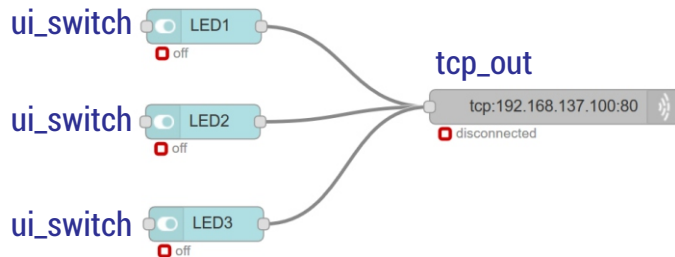
}

}
```



ATIVIDADE 3C - UTILIZANDO O NODE-RED

Nesta atividade, iremos aproveitar o hardware e software da Atividade 3B para acionarmos os LEDs através de um *dashboard* do Node-RED. Assim, monte seu fluxo no Node-RED, conforme a figura abaixo:



CONFIGURAÇÕES DOS ELEMENTOS

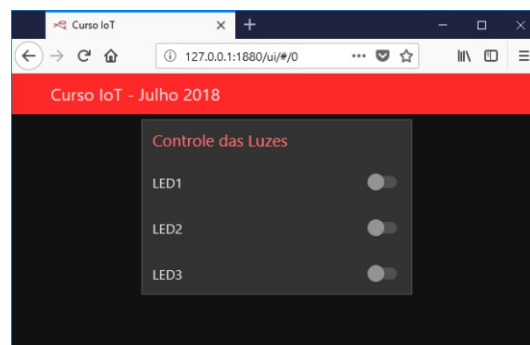
LED1 - ui_switch

LED2 - ui_switch

LED3 - ui_switch

tcp_out

DASHBOARD



Ao clicar no switch de cada LED, o comando LED=ON/LED=OFF será enviado ao ESP-01 via socket e o LED correspondente deverá acender e desligar.



O QUE É MQTT?

Message Queue Telemetry Transport (MQTT) é um protocolo aberto Machine-to-Machine (M2M), que foi inventado em 1999, e esta se tornando um dos padrões de protocolos para a Internet das Coisas (IoT) mais utilizados. MQTT foi desenvolvido por Andy Stanford-Clark (IBM) e Arlen Nipper (Arcom, now Cirrus Link) com o objetivo de ser um protocolo com mínimas perdas energéticas e uso mínimo de banda de conexões através de satélites. As principais características do protocolo são:

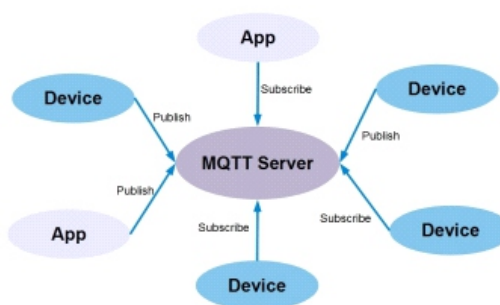
- Simplificidade de implementação
- Disponibilidade de "Quality of Service Data Delivery"
- Baixo consumo de energia e de dados

Servidores MQTT:

- Apollo
- JoramMQ
- Mosquitto
- RabbitMQ

Por isso se tornou uma ótima opção para uso com Internet das Coisas. Neste curso iremos utilizar o servidor Mosquitto, pelas seguintes razões:

- Open source MQTT broker;
- Biblioteca em C disponível;
- Vasta documentação disponível;
- Suporte a TLS / SSL;
- Boa performance;
- Suporta Qos0, Qos1 e Qos2



MQTT: PUB/SUB

Clientes conectam ao "Broker" e assinam os tópicos, por exemplo:

```
client.subscribe('toggleLight/1')
client.subscribe('toggleLight/2')
client.subscribe('toggleLight/3')
```

Clientes podem publicar mensagens nos tópicos:

```
client.publish('toggleLight/1', 'toggle');
client.publish('toggleLight/2', 'toggle');
```

Todos os clientes receberão todas as mensagens publicadas nos tópicos que estão "assinando". Mensagens podem ser, por exemplo: Textos, Imagens, etc...



BOTÃO DE PAINEL PROGRAMÁVEL NA NUVEM



Fonte: Amazon

<https://aws.amazon.com/pt/iotbutton/>

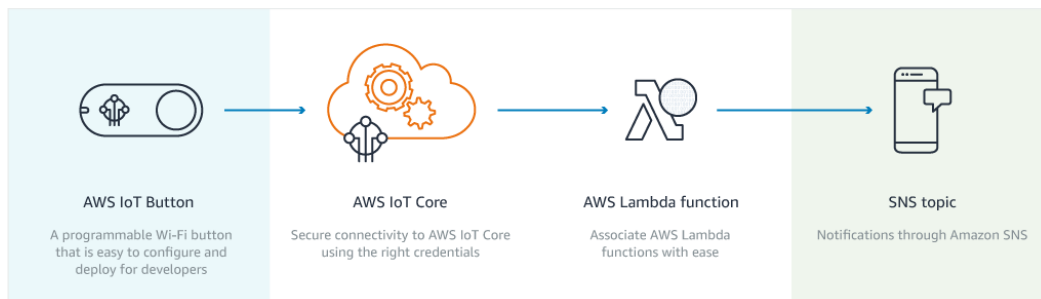
O AWS IoT Button é um botão programável baseado no hardware do Amazon Dash Button. Este dispositivo simples de Wi-Fi é fácil de configurar e foi projetado para permitir que os desenvolvedores comecem a usar os serviços AWS IoT Core, AWS Lambda, Amazon DynamoDB, Amazon SNS e muitos outros serviços da Amazon Web Services sem necessidade de escrever código específico para dispositivos.

Você pode codificar a lógica do botão na nuvem configurando os cliques desse botão para contar ou rastrear itens, ligar para uma pessoa ou alertar alguém, iniciar ou parar uma atividade, solicitar serviços ou, até mesmo, fornecer opinião. Por exemplo, você pode clicar no botão para destravar ou dar a partida em um carro, abrir a porta da garagem, chamar um táxi, ligar para um companheiro ou representante de serviço ao cliente, rastrear o uso de produtos de consumo domésticos ou medicamentos, ou ainda controlar remotamente a operação de eletrodomésticos.

O botão pode ser usado como um controle remoto para o Netflix, um interruptor para sua lâmpada Philips Hue, um dispositivo de registro/reserva para clientes do Airbnb ou uma maneira de pedir sua pizza favorita para entrega. Você pode integrá-lo a APIs de terceiros como Twitter, Facebook, Twilio, Slack ou, até mesmo, aplicativos da sua própria empresa. Conecte-o a coisas que nunca tinha imaginado antes. Estamos ansiosos para ver o que você vai criar usando o AWS IoT Button.

Se você quiser usar dispositivos simples que se conectam ao AWS IoT Core sem necessidade de configuração, use o serviço AWS IoT 1-Click.

COMO FUNCIONA



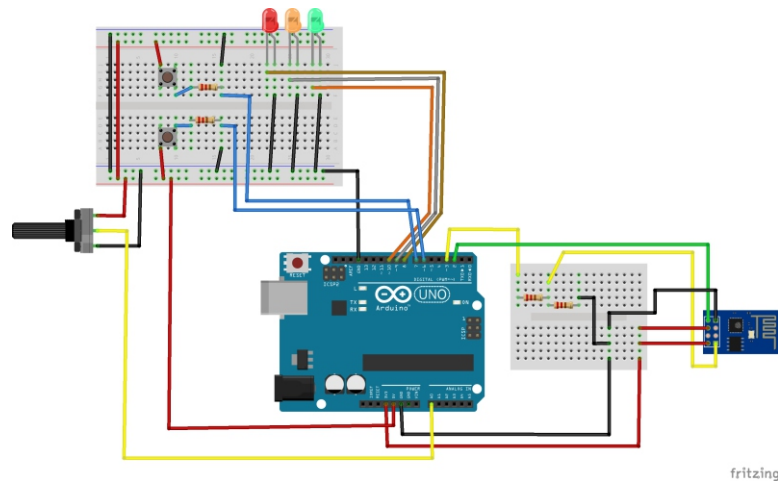
Fonte: Amazon

<https://aws.amazon.com/pt/iotbutton/>



ATIVIDADE 3D - UTILIZANDO O NODE-RED + MQTT

Agora, aproveitaremos o hardware da Atividade 3C e adicionaremos um potenciômetro na porta analógica A0 do Arduino e 2 botões nas portas 6 e 7. Nesta atividade, criaremos um *dashboard* no Node-Red, que além de se comunicar com o Arduino+ESP01 via sockets, também irá publicar e assinar tópicos em um servidor MQTT. Segue o esquemático eletrônico.



CÓDIGO FONTE

```
/*
ATIVIDADE 3D - Node-RED - Serversocket + MQTT - Enviando dados sensor analógico
CURSO IoT - Julho 2018
Internet das Coisas com Arduino e ESP-01
Autor: Prof. Breno M. de Oliveira
*/

#include <Arduino.h>
#include <SoftwareSerial.h> //Biblioteca para criação de portas seriais (softserial)

//Declarando os pinos dos LEDs, botões e potenciômetro
#define pin_led1 8
#define pin_led2 9
#define pin_led3 10
#define pin_btn1 6
#define pin_btn2 7
#define pin_pot A0

SoftwareSerial ESP01(2, 3); //Cria uma nova interface serial através dos Pinos D2 e D3
do Arduino,
//onde será conectado o módulo wifi ESP-01
// Pino 2 é o RX do Arduino (TX do ESP)
// Pino 3 é o TX do Arduino (RX do ESP) - Precisa do
divisor de tensão 5V -> 3.3V
```

CONTINUA...



CONTINUAÇÃO

```
// Função para envio de comandos ATs para o ESP-01
// Parametros de Entrada: Cmd - Comando , ACK - retorno
// Parametros de Saída: Verdadeiro ou Falso (true or false)
boolean EnviaComandoESP01(String cmd, String ack, unsigned long timeout, String
descricao, String retorno)
{
    Serial.println(cmd);
    ESP01.println(cmd); // Envia o comando (cmd) para o ESP01
    timeout = timeout + millis();
    while (millis() < timeout)
    {
        if (ESP01.available())
        {
            String resposta = ESP01.readString();
            resposta.trim();
            if (resposta.indexOf(ack) != -1)
            {
                if (ack == "+CIFSR:STAIP,")
                {
                    int i1 = resposta.indexOf("+CIFSR:STAIP,") + 13; //13 é o tamanho de
"+CIFSR:STAIP,"
                    int i2 = resposta.indexOf("+", i1 + 2);
                    Serial.print("IP:");
                    String ip = resposta.substring(i1, i2);
                    ip.trim();
                    Serial.println(ip);
                    return true;
                }
                Serial.print(descricao);
                Serial.print(" : ");
                Serial.println(retorno);
                return true;
            }
        }
    }
    return false;
}

String cipSend = "";
String temp = "";
String StringEntrada = ""; //Armazena as Strings vindas do Wifi
String status = "";

void setup()
{
    pinMode(pin_led1, OUTPUT);
    pinMode(pin_led2, OUTPUT);
    pinMode(pin_led3, OUTPUT);
    pinMode(pin_btn1, OUTPUT);
    pinMode(pin_btn2, OUTPUT);
    Serial.begin(9600);
    ESP01.begin(9600);
    EnviaComandoESP01("AT+RST", "Ready", 5000, "ESP01 - Resetando o ESP01", "Reset OK");
    EnviaComandoESP01("AT+CWQAP", "OK", 5000, "ESP01 - Desconectando de Redes
Anteriores...", "Desconectado!");
    EnviaComandoESP01("AT+CWMODE=1", "OK", 5000, "ESP01 - Setando Modo Station...",
"Setado!");
}
```

CONTINUA...



CONTINUAÇÃO

```
if (EnviaComandoESP01("AT+CWJAP=\"CursoIoT\\", \"julho2018\\", "WIFI GOT IP", 20000,
"ESP01 - Conectando na Rede CursoIoT...", "Conectado!"))
{
    EnviaComandoESP01("AT+CIFSR", "+CIFSR:STAIP,", 5000, "ESP01 - Adquirindo numero de
IP...", "IP Adquirido:");
    EnviaComandoESP01("AT+CIPMUX=1", "OK", 5000, "ESP01 - Setando TCPServer para
multiplas conexoes (Max 5) ...", "TCPServer Multiplicas Conexoes OK");
    EnviaComandoESP01("AT+CIPSERVER=1,80", "OK", 5000, "ESP01 - Startando o TCPServer
na porta 80...", "TCPServer Operante");
}
else
{
    Serial.println("Nao foi possível conectar a rede Wifi!");
}
}

void loop(){

    while (ESP01.available())
    {
        StringEntrada = ESP01.readString();
        Serial.println(StringEntrada);

        StringEntrada.trim(); // Remove caracteres fantasmas

        if (StringEntrada.indexOf("LED1=ON") != -1)
        { //Se encontrar a String "LED1=ON" dentro da StringEntrada
            digitalWrite(pin_led1, HIGH);
            Serial.println("LED1 - Aceso");
        }

        if (StringEntrada.indexOf("LED2=ON") != -1)
        {
            digitalWrite(pin_led2, HIGH);
            Serial.println("LED2 - Aceso");
        }

        if (StringEntrada.indexOf("LED3=ON") != -1)
        {
            digitalWrite(pin_led3, HIGH);
            Serial.println("LED3 - Aceso");
        }

        if (StringEntrada.indexOf("LED1=OFF") != -1)
        {
            digitalWrite(pin_led1, LOW);
            Serial.println("LED1 - Apagado");
        }

        if (StringEntrada.indexOf("LED2=OFF") != -1)
        {
            digitalWrite(pin_led2, LOW);
            Serial.println("LED2 - Apagado");
        }

        if (StringEntrada.indexOf("LED3=OFF") != -1)
        {
            digitalWrite(pin_led3, LOW);
            Serial.println("LED3 - Apagado");
        }
    }
}
```

CONTINUA...



CONTINUAÇÃO

```
if (StringEntrada.indexOf("TEMP=?") != -1)
{
    temp = String(analogRead(A0));
    cipSend = "AT+CIPSEND=";
    cipSend += 0;
    cipSend += ",";
    cipSend += temp.length();
    cipSend += "\r\n";
    EnviaComandoESP01(cipSend, "SEND OK", 2000, "ESP01 - Enviando temperatura...",
"Enviada");
    delay(20);
    ESP01.print(temp);
}

if (StringEntrada.indexOf("BTN1=?") != -1)
{
    if (digitalRead(pin_btn1))
    {
        status = "BTN1=ON";
    }
    else
    {
        status = "BTN1=OFF";
    }
    cipSend = "AT+CIPSEND=";
    cipSend += 0;
    cipSend += ",";
    cipSend += status.length();
    cipSend += "\r\n";
    EnviaComandoESP01(cipSend, "SEND OK", 2000, "ESP01 - Enviando estado do Botão
1...", "Enviado");
    delay(20);
    ESP01.print(status);
    //EnviaComandoESP01("AT+CIPCLOSE=0","OK",2000,"ESP01 - Fechando","Enviado");
}

if (StringEntrada.indexOf("BTN2=?") != -1)
{
    if (digitalRead(pin_btn2))
    {
        status = "BTN2=ON";
    }
    else
    {
        status = "BTN2=OFF";
    }
    cipSend = "AT+CIPSEND=";
    cipSend += 0;
    cipSend += ",";
    cipSend += status.length();
    cipSend += "\r\n";
    EnviaComandoESP01(cipSend, "SEND OK", 2000, "ESP01 - Enviando estado do Botão
2...", "Enviado");
    delay(20);
    ESP01.print(status);
}
}
```

CONTINUA...



Atividade 3 - Parte D

Group: Botões [Curso IoT - Julho 2018]

Size: auto

Icon: optional icon

Label: Botão 1?

Colour: optional text/icon color

Background: optional background color

When clicked, send:

Payload:

Topic: /botoes/btn1/

☐ If `msg` arrives on input, pass through to output:

Name:

Botao 1? - ui_button

Group: Botões [Curso IoT - Julho 2018]

Size: auto

Icon: optional icon

Label: Botão 2?

Colour: optional text/icon color

Background: optional background color

When clicked, send:

Payload:

Topic: /botoes/btn2/

☐ If `msg` arrives on input, pass through to output:

Name:

Botão 2? - ui_button

IP do ESP01

Server: port: 80

Return: never - keep connection open

Name:

tcp:192.168... - tcp_request

Server: Curso2018

Topic: /leds/+/

QoS: 0

Name:

/leds/+/ - mqtt in

Server: Curso2018

Topic:

QoS: 0

Retain: ☐

Name:

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

mqtt - mqtt out

Name: Curso2018

Connection: Security Messages

Server: 54.84.101.243 Port: 1883

☐ Enable secure (SSL/TLS) connection

Client ID: NR

☐ Keep alive time (s): 60 ☐ Use clean session

☒ Use legacy MQTT 3.1 support

configurações servidor MQTT

Server: Curso2018

Topic: /temperatura/

QoS: 0

Name:

/temperatura/ - mqtt in

Group: Mensagens [Curso IoT - Julho 2018]

Size: auto

Label: Tópicos:

Value format: {{msg.topic}}

Layout:

Name:

Tópicos - ui_text



Server: Curso2018

Topic: /botoes/btn1/

QoS: 0 Retain: ☐

Name: Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

/botoes/btn1/- mqtt out

Server: Curso2018

Topic: /botoes/btn2/

QoS: 0 Retain: ☐

Name: Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

/botoes/btn2/- mqtt out

Server: Curso2018

Topic: /temperatura/

QoS: 0 Retain: ☐

Name: Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

/temperatura/- mqtt out

Name: AnalisaDados

Function:

```
1 if (msg.topic === "/botoes/btn1/") {
2   return [msg, null, null];
3 }
4
5 if (msg.topic === "/botoes/btn2/") {
6   return [null, msg, null];
7 }
8 if (msg.topic === "/temperatura/") {
9   return [null, null, msg];
10 }
11
12
13
```

Outputs: 3

AnalisaDados - function

Name: String1

From: msg. payload

Methods:

To: msg. payload

String1- string



Atividade 3 - Parte D

Group: Temperatura [Curso IoT - Julho 2018]

Size: auto

Type: Donut

Label: Temperatura

Value format: {{value | number:1}}%

Units: units

Range: min 0 max 1023

Colour gradient: [Green, Yellow, Red]

Sectors: 0 optional optional 1023

Name:

Temperatura- ui_gauge

Group: Temperatura [Curso IoT - Julho 2018]

Size: auto

Label: Gráfico de Temperatura

Type: Line chart ☐ enlarge points

X-axis: last 1 hours OR 1000 points

X-axis Label: HH:mm:ss

Y-axis: min max

Legend: None Interpolate linear

Series Colours: [Blue, Green, Red, Orange, Purple, Yellow]

Blank label: display this text before valid data arrives

☐ Use deprecated (pre 2.5.0) data format.

Name:

Gráfico de temp...- ui_chart

Name: String2

From: msg.payload

Methods:

To: msg.payload

String2 - string

Name: Converte Falas

Function:

```
1 if (msg.payload === "LED1=ON") {
2   msg.payload = "LED 1 - Ligado"
3   return msg
4 }
5
6 if (msg.payload === "LED2=ON") {
7   msg.payload = "LED 2 - Ligado"
8   return msg
9 }
10
11 if (msg.payload === "LED3=ON") {
12   msg.payload = "LED 3 - Ligado"
13   return msg
14 }
15
16 if (msg.payload === "LED1=OFF") {
17   msg.payload = "LED 1 - Desligado"
18   return msg
19 }
20
21 if (msg.payload === "LED2=OFF") {
22   msg.payload = "LED 2 - Desligado"
23   return msg
24 }
25
26 if (msg.payload === "LED3=OFF") {
27   msg.payload = "LED 3 - Desligado"
28   return msg
29 }
```

Outputs: 1

ConverteFalas - function



Atividade 3 - Parte D

Name

Converte Falas

Function

```
1 * if (msg.payload === "BTN1=OFF" ) {
2   msg.payload = "Botão 1,Não Pressionado"
3   return msg
4 * }
5
6 * if (msg.payload === "BTN2=OFF" ) {
7   msg.payload = "Botão 2,Não Pressionado"
8   return msg
9 * }
10
11 * if (msg.payload === "BTN1=ON" ) {
12   msg.payload = "Botão 1,Pressionado"
13   return msg
14 * }
15
16 * if (msg.payload === "BTN2=ON" ) {
17   msg.payload = "Botão 2,Pressionado"
18   return msg
19 * }
20
21
```

Outputs 1

ConverteFalas - function

Group: Controle das Luzes [Curso IoT - Julho 2018]

TTS Voice: 0 : Microsoft Maria Desktop - Portuguese(Brazil) (pt)

☒ Play audio when window not in focus.

Name: Name

audio out - ui_audio

Group: Controle das Luzes [Curso IoT - Julho]

Size: auto

Label: Retorno ESP01

Value format: {{msg.payload}}

Layout: label value, labelvalue, labelvalue, label value, label value

Name:

Retorno ESP01 - ui_text

Group: Botões [Curso IoT - Julho 2018]

Size: auto

Label: Retorno ESP01

Value format: {{msg.payload}}

Layout: label value, labelvalue, labelvalue, label value, label value

Name:

Retorno ESP01 - ui_text



- DIOGENES, Yuri. Certificação Cisco. Rio de Janeiro: Axcel Books, 2004.
- COMER, Douglas ; STEVENS, David L. Interligação em Rede com TCP/IP. Rio de Janeiro: Campus, 1999.
- MORIMOTO, Carlos E. Guia Completo de Redes. Guia do Hardware. 2ª Ed., Editora GDH Press e Sul.
- FILIPPETTI, Marcos A. CCNA 3.0 – Guia Completo de Estudo. Florianópolis: Visual Books, 2002.

Sites:

- Arquitetura de Redes: <https://www.uniaogeek.com.br/arquitetura-de-redes-tcpip/> - Acessado em 18/07/2018
- ROOM-15 - <https://room-15.github.io/blog/2015/03/26/esp8266-at-command-reference/>
- Espressif - <https://www.espressif.com/>
- Arduino - www.arduino.cc